

Capítulo

1

Inteligência Artificial aplicada à Resposta a Incidentes

Rodrigo Sanches Miani e Sebastião Alves de Jesus Filho (UFU); Diego Kreutz e Cristhian Kapelinski (UNIPAMPA); Lourenço Alves Pereira Júnior, Sidnei Barbieri e Murray Evangelista de Souza (ITA)

Abstract

Incident response (IR) teams face a volume of notifications that has long outgrown what manual triage can absorb. This chapter examines how Artificial Intelligence (AI), and Large Language Models in particular, can support incident response without displacing the human analyst. We combine consolidated frameworks (NIST SP 800-61r3, NIST CSF 2.0, and the FIRST CSIRT Services Framework), a structured review of the recent literature organized by application category, and three hands-on case studies conducted over a base of real, previously pseudonymized incident tickets provided by partner CSIRTs: pseudonymization of tickets, automated incident classification, and assisted playbook generation. Each case study pairs a demonstration of the proponents' tools with a small, self-contained and executable notebook, published in an open repository. The chapter's central message is that AI does not replace the incident response process; it enlarges triage, analysis, documentation, and recommendation capacity when it is integrated with consolidated frameworks, trustworthy data, adequate anonymization, and human validation.

Resumo

Equipes de Resposta a Incidentes (RI) convivem com um volume de notificações que há muito ultrapassou a capacidade de triagem manual. Este capítulo examina como a Inteligência Artificial, e, em especial, os Modelos de Linguagem de Grande Escala, podem apoiar a resposta a incidentes sem deslocar o analista humano. Combinam-se frameworks consolidados (NIST SP 800-61 Rev. 3, NIST CSF 2.0 e FIRST CSIRT Services Framework), uma revisão da literatura recente, organizada por categoria de aplicação, e três estudos de caso conduzidos com base em uma base de tickets reais de incidentes, previamente pseudonimizados, cedidos por CSIRTs parceiros: pseudonimização de tickets,

classificação automática de incidentes e geração assistida de playbooks. Cada estudo de caso combina a demonstração das ferramentas do grupo proponente com um notebook simples, autocontido e executável, publicado em um repositório aberto. A mensagem central do capítulo é que a IA não substitui o processo de resposta a incidentes, mas amplia a capacidade de triagem, análise, documentação e recomendação quando integrada a frameworks consolidados, dados confiáveis, anonimização adequada e validação humana.

1.1. Introdução

A resposta a incidentes (RI) deixou de ser uma função predominantemente reativa das equipes de segurança para se tornar uma operação contínua e sob pressão permanente. Dois indicadores tornam essa mudança concreta. O primeiro é o volume de vulnerabilidades publicadas: o número de *Common Vulnerabilities and Exposures* (CVEs) divulgadas por ano cresceu de aproximadamente 7.928 em 2014 para 40.287 em 2024, atingiu 48.448 em 2025 e tem previsão mediana de 59.400 para 2026 [CVE26]. O segundo é o volume de notificações efetivamente recebidas pelas equipes nacionais: somente em 2024, o CERT.br registrou 516.556 notificações de incidentes, o que equivale a uma média de 1.411 por dia [CER25]. Esses dois indicadores medem coisas distintas, superfície de ataque potencial e eventos reportados, mas apontam na mesma direção: o trabalho de triagem cresce mais rápido do que a capacidade de contratar e formar analistas, cujo déficit global é estimado em mais de quatro milhões de profissionais [ISC23].

A consequência operacional desse descompasso é conhecida na literatura como *alert fatigue*. O estudo qualitativo de Alahmadi et al. [AAM22], conduzido com analistas de centros de operações de segurança (*Security Operations Centers*, SOC), é uma das principais referências sobre o tema e mostra que o problema não decorre apenas do volume, mas também da qualidade dos alertas. Os autores distinguem *falso positivo* de *alerta benigno*, uma diferença claramente percebida pelos analistas, mas raramente representada pelas ferramentas, e identificam cinco propriedades que um alerta deve apresentar para ser acionável: ser confiável, explicável, analítico, contextual e transferível. Alertas que falham nessas propriedades produzem um efeito perverso: o analista aprende a desconsiderá-los, e a desensibilização passa a ser um risco de segurança em si.

A natureza recorrente e automatizada de muitas ameaças amplia a carga operacional sobre as equipes de segurança. Um levantamento setorial que conduzimos para o setor de telecomunicações [PJS25a] mostra um cenário dominado por tentativas repetitivas e automatizadas: tentativas excessivas de conexão SSH, varreduras de portas TCP, exploração de falhas em equipamentos de borda como *gateways* residenciais (por exemplo, CVE-2017-17215 e CVE-2014-8361), amplificação de DNS, túneis de DNS e *botnets* de IoT derivadas do Mirai. Sobre essa base ruidosa, atuam grupos persistentes com objetivos específicos relacionados à infraestrutura de telecomunicações, entre eles LightBasin e Salt Typhoon [PJS25a]. Esse perfil dialoga diretamente com a base de *tickets* usada neste minicurso, na qual a exploração de vulnerabilidade, a tentativa de intrusão e a negação de serviço concentram a maior parte dos registros (Seção 1.4.3).

O quadro que emerge, portanto, não é o de uma equipe que precisa de mais ferramentas de detecção, mas o de uma equipe que precisa de apoio para *decidir* sobre o que já

foi detectado. É exatamente nesse ponto que este capítulo posiciona o uso de Inteligência Artificial.

1.1.1. Resposta a incidentes na gestão de risco cibernético

Antes de discutir as técnicas aplicadas à resposta a incidentes, é importante estabelecer o contexto conceitual em que ela se insere. A revisão 3 do NIST SP 800-61 [NRSS25] promoveu uma mudança de perspectiva relevante: em vez de tratar a resposta a incidentes como um processo autocontido com fases próprias, a norma passa a apresentá-la como um *perfil comunitário* do NIST Cybersecurity Framework 2.0 [NIS24]. Na prática, isso significa que as atividades de resposta são distribuídas entre as seis funções do CSF (Governar, Identificar, Proteger, Detectar, Responder e Recuperar) e que a preparação e o aprendizado pós-incidente deixam de ser etapas de um ciclo fechado para se tornarem atividades contínuas de gestão de risco.

Essa reformulação tem uma consequência prática para o tema deste capítulo. Se a resposta a incidentes é um recorte da gestão de risco, a pergunta não é “onde encaixar a IA no ciclo de RI”, mas “quais funções do CSF admitem apoio automatizado, em que grau e sob que supervisão”. Em trabalho anterior do grupo proponente [PJSS⁺25], respondemos a essa pergunta ao decompor um SOC em treze funções e atribuir a cada uma um nível de automação, questão retomada em detalhe na Seção 1.8.4.

1.1.2. De onde vem a IA em segurança

O uso de aprendizado de máquina em segurança é anterior à onda atual de modelos generativos por cerca de duas décadas. Uma revisão sistemática que realizamos [PJS25b] percorre essa trajetória ao longo de três gerações. A primeira concentra-se na detecção de intrusão e de anomalias, com classificadores clássicos (árvores de decisão, florestas aleatórias, SVM) sobre *features* extraídas do tráfego de rede. A segunda substitui a engenharia manual de *features* por aprendizado profundo (CNN, LSTM, GRU, autocodificadores e, mais recentemente, *transformers*) e traz consigo dois problemas herdados: o desbalanceamento severo das bases de dados de intrusão e a opacidade dos modelos. A terceira, iniciada em 2023, é a da IA generativa, cujo impacto o relatório caracteriza como duplo: amplia a capacidade defensiva e, simultaneamente, reduz o custo de produção de artefatos ofensivos.

Para os propósitos deste capítulo, interessa em particular o subconjunto de aplicações que operam sobre *texto* produzido por humanos ou por ferramentas, e não sobre fluxos de pacotes. É nesse subconjunto que os modelos de linguagem apresentam vantagem estrutural, pois *tickets*, relatos, *logs* e relatórios são exatamente o tipo de entrada não estruturada que os classificadores tradicionais têm dificuldade em tratar. A revisão da Seção 1.3 organiza esse espaço em seis categorias: triagem e classificação de incidentes, sistemas de recomendação e apoio à decisão, sumarização e geração de artefatos de defesa, explicabilidade e consciência situacional, IA agêntica e automação de SOCs, e estado da prática industrial.

1.1.3. O diferencial do minicurso: dados reais anonimizados

A maior parte do material didático disponível sobre IA aplicada à segurança baseia-se em exemplos sintéticos. Isso é compreensível, já que dados de incidentes são sensíveis por natureza, mas cobra um alto preço pedagógico: exemplos sintéticos são limpos, sem ambiguidade e sem ruído e, por isso, escondem justamente as dificuldades que caracterizam a tarefa real. Este minicurso adota um compromisso metodológico distinto. Os três estudos de caso são conduzidos com base em *tickets* de incidentes reais, cedidos por CSIRTs parceiros e previamente pseudonimizados com as ferramentas da linha AnonLFI/AnonShield [BEG⁺25, KLMK25, KLM⁺26]. Esse compromisso não é gratuito: ele impõe uma política de dados explícita, descrita na Seção 1.4.3, segundo a qual os dados brutos nunca deixam a organização de origem, a anonimização é executada localmente e *offline*, e as demonstrações que dependem de serviços externos pagos são apresentadas pré-executadas sobre dados pseudonimizados ou sintéticos.

A origem dos dados também não é acidental. Ela decorre do ecossistema do GT-LFI (*Learn From Incidents*) [GT-25b], projeto do programa Hackers do Bem da RNP que parte de um diagnóstico convergente com o argumento inicial deste capítulo: os cursos tradicionais de formação em segurança priorizam conteúdos conceituais e exercícios descontextualizados, o que produz profissionais sem experiência prática suficiente para reconhecer padrões, classificar incidentes com precisão ou conduzir respostas sob pressão. A resposta do GT-LFI a esse diagnóstico é converter incidentes reais, depois de anonimizados, em insumo simultaneamente operacional e pedagógico.

1.1.4. Organização do capítulo e roteiro das práticas

O capítulo está organizado em oito seções. A Seção 1.2 nivela conceitos e *frameworks*, incluindo os fundamentos de privacidade e a taxonomia de incidentes que são pré-requisitos dos estudos de caso. A Seção 1.3 apresenta o estado da arte, organizado em seis categorias de aplicação mencionadas acima. A Seção 1.4 caracteriza os ambientes, as ferramentas e a base de dados em que os estudos de caso operam.

As Seções 1.5, 1.6 e 1.7 apresentam os três estudos de caso. Eles não são exercícios independentes: formam um *pipeline* único, no qual a saída de um alimenta o seguinte, espelhando o ciclo pelo qual o ecossistema GT-LFI opera em produção. O *ticket* é primeiro pseudonimizado (EC1, módulo ANON-LFI), depois classificado (EC2, módulo AutoClass-LFI) e, finalmente, convertido em recomendação estruturada de resposta (EC3, também AutoClass-LFI), com validação humana por analistas de Pontos de Presença da RNP, o que fecha o ciclo (Módulo PoP). A Seção 1.8 consolida os desafios, as tendências e as conclusões. A Figura 1.1 resume esse encadeamento, com o artefato que cada etapa entrega à seguinte e o ponto de validação humana em cada transição.

Cada estudo de caso tem uma parte prática em dois tempos. No primeiro, demonstra-se a ferramenta do grupo proponente em operação, o AnonShield na linha de comando e o AutoClass-LFI em sua interface Web. No segundo, os participantes executam um *notebook* simples, didático e autocontido, que reimplementa o essencial do mecanismo em poucas dezenas de linhas, de modo que o comportamento observado na ferramenta se torne inspecionável. Um overview do minicurso pode ser visto no site <https://sbseg26-mcl.github.io>, incluindo referências para repositórios,

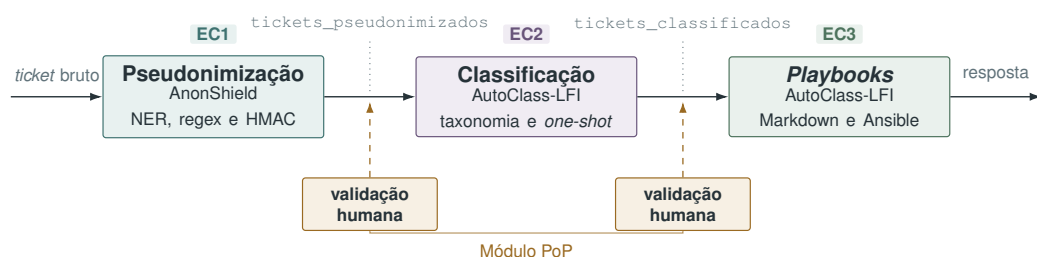


Figura 1.1. Pipeline dos três estudos de caso. A saída de cada etapa é a entrada da seguinte, e a validação humana é barreira obrigatória em cada transição, não apenas ao final.

exemplos, entre outras coisas. Os *notebooks*, os dados sintéticos, os *prompts* e os *playbooks* de referência não estão reproduzidos neste capítulo: estão disponíveis no repositório público do minicurso, em <https://github.com/orgs/sbseg26-mc1> [SBS26].

1.2. Conceitos Básicos e Frameworks

Esta seção define o vocabulário comum do capítulo. Ela cobre o ciclo de vida da resposta a incidentes e sua integração à gestão de risco (Seção 1.2.1), a organização de CSIRTs e SOCs, incluindo uma arquitetura de referência (Seção 1.2.2), o papel das taxonomias (Seção 1.2.3), os fundamentos de IA necessários aos estudos de caso (Seção 1.2.4) e os fundamentos de privacidade e proteção de dados (Seção 1.2.5). As duas últimas subseções são pré-requisitos diretos para os estudos de caso 1 e 2.

1.2.1. O ciclo de vida da resposta a incidentes

O NIST SP 800-61 Rev. 3 [NRSS25] substitui a estrutura em quatro fases da revisão anterior por um perfil comunitário do NIST CSF 2.0 [NIS24]. As seis funções do CSF passam a organizar as atividades de resposta:

Governar (GV). Estabelece a estratégia, os papéis, as políticas e a supervisão do programa de segurança. É a função responsável pela existência de um plano de resposta, pela definição de responsabilidades e pelos acordos com terceiros. No contexto deste capítulo, é também a função que decide *quanta autonomia* delegar a um sistema automatizado.

Identificar (ID). Compreende o inventário de ativos, a avaliação de risco e, de forma central para a RI, a melhoria contínua com base nas lições aprendidas. É aqui que o aprendizado pós-incidente deixa de ser uma etapa terminal do ciclo e passa a alimentar a gestão de risco.

Proteger (PR). Controles preventivos: gestão de identidade, proteção de dados, resiliência da infraestrutura.

Detectar (DE). Monitoramento contínuo e análise de eventos adversos. É a função que produz o volume de alertas discutido na Seção 1.1 e, portanto, a que mais se beneficia de redução de falsos positivos.

Responder (RS). Gestão do incidente: análise, triagem, categorização, notificação das partes interessadas e mitigação. É o núcleo operacional dos três estudos de caso.

Recuperar (RC). Restauração dos serviços e comunicação da recuperação.

A comparação entre o *Initial Public Draft* [NRSS24] e a versão final [NRSS25] explicita o que a comunidade pediu durante a consulta pública: maior ênfase na articulação com o CSF e menor prescrição de processo. A norma final oferece um perfil de referência, não um manual de operação, e deixa deliberadamente a cada organização a decisão sobre como instanciá-lo.

A Tabela 1.1 indica, para cada função, os pontos em que as técnicas de IA podem oferecer apoio e os que permanecem sob decisão humana. Essa distinção é retomada quantitativamente na Seção 1.8.4.

Tabela 1.1. Pontos de apoio de IA por função do NIST CSF 2.0, no recorte da Resposta a incidentes.

Função	Apoio possível de IA	Permanece humano
Governar	Sumarização de políticas; verificação de conformidade documental	Definição de risco aceitável e de níveis de autonomia
Identificar	Extração estruturada de lições aprendidas; correlação histórica	Avaliação de risco de negócio
Proteger	Geração de regras (por exemplo, WAF) e revisão de configuração	Homologação e aplicação em produção
Detectar	Redução de falsos positivos; priorização; enriquecimento	Definição de limiares e critérios de severidade
Responder	Triagem, classificação, sumarização, geração de <i>playbooks</i>	Autorização e execução de ações de mitigação
Recuperar	Redação de comunicados e relatórios pós-incidente	Declaração de recuperação e comunicação oficial

1.2.2. CSIRTs e SOC: serviços, estruturas e arquitetura

O FIRST CSIRT Services Framework v2.1 [FIR19] organiza a atuação de uma equipe de resposta em áreas de serviço. Para os fins deste capítulo, as mais relevantes são gestão de eventos de segurança (monitoramento, detecção, triagem e análise), gestão de incidentes (recepção do relato, análise, coordenação, mitigação e recuperação) e gestão de vulnerabilidades. O *framework* é deliberadamente agnóstico quanto às ferramentas: descreve *o que* uma equipe faz, não *como*.

A distinção entre CSIRT e SOC é frequentemente fonte de confusão. De forma operacional, um SOC é uma estrutura de monitoramento contínuo, tipicamente ancorada em uma plataforma de gestão de informações e eventos de segurança (*Security Information and Event Management*, SIEM), cuja função primária é detectar e triar; um CSIRT é uma estrutura de coordenação de resposta, que pode atender a uma organização (CSIRT interno) ou a uma comunidade (CSIRT de coordenação, como os PoPs da RNP ou o próprio CERT.br). Muitas organizações combinam as duas funções na mesma equipe; outras mantêm o SOC terceirizado e o CSIRT interno. O ponto relevante para este capítulo é que ambos consomem o mesmo insumo, isto é, relatos textuais de eventos adversos, e enfrentam o mesmo gargalo de triagem.

Arquitetura de referência de um SOC

Para relacionar as técnicas discutidas adiante às diferentes etapas da resposta a incidentes, adotamos uma arquitetura de referência. Adotamos aqui a arquitetura de referência que propusemos em [PJSS⁺25], organizada em treze funções, segundo os escopos *Detect* e *Respond* do NIST CSF 2.0. Em ordem de fluxo de dados, são elas: sensores de telemetria (ST); agentes de coleta (AC); ingestão em fila de mensagens (IG); normalização de dados (ND); armazenamento e indexação (AI); análise em tempo real e correlação (AT); detecção avançada (DA); aprendizado de máquina (ML); visualização e *dashboards* (VD); integração de IA com modelos de linguagem (IA); orquestração e execução de *playbooks* via SOAR (SO); *threat intelligence* (TI); e equipes e operação (EO). Essas siglas são as usadas nas Tabelas 1.3 e 1.9.

Os três estudos de caso deste capítulo situam-se na função de integração de IA (IA), com o estudo de caso 3 alcançando também a função de orquestração (SO) e todos os três dependendo criticamente da função de equipes e operação (EO), que é onde a validação humana ocorre. O estudo de caso 1 atua como pré-condição de conformidade: sem ele, a função IA não pode existir sem violar a política de dados da organização. A Figura 1.2 resume essa arquitetura e posiciona os estudos de caso sobre ela.

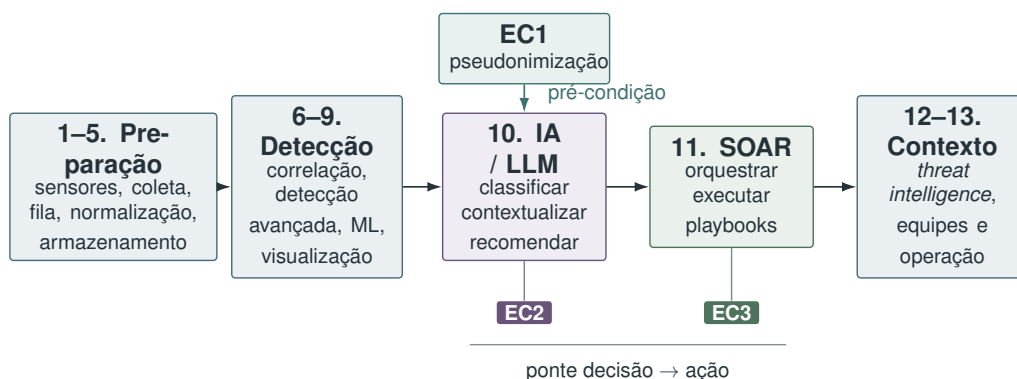


Figura 1.2. Arquitetura de referência de um SOC em treze funções, com a posição dos três estudos de caso e a pseudonimização como pré-condição de conformidade. Adaptada de [PJSS⁺25].

1.2.3. Taxonomias e vocabulários padronizados

Classificar um incidente exige, antes de tudo, um vocabulário. A ausência de padronização terminológica entre organizações é apontada de forma recorrente como obstáculo à automação [Ibr19, SLK⁺25]: relatos que descrevem o mesmo fenômeno recebem rótulos distintos, o que impede tanto a agregação de métricas quanto o treinamento de modelos supervisionados.

Há várias taxonomias em uso. O MITRE ATT&CK [MIT25] cataloga táticas e técnicas adversárias observadas em incidentes reais e é o vocabulário de referência para *threat intelligence*, mas descreve *como* um adversário age, não *que tipo de incidente* ocorreu. A taxonomia de referência da ENISA [ENI18] foi concebida para harmonizar a comunicação entre os CSIRTs europeus. A taxonomia do CERT.br, usada nas notificações nacionais, adota um conjunto compacto de categorias operacionais. O NIST SP

800-61r3 [NRSS25], por sua vez, não impõe um padrão rígido, mas sugere categorias funcionais adaptáveis a cada instituição.

Neste capítulo, adota-se como referência principal o conjunto de doze categorias funcionais derivadas do NIST SP 800-61r3 e consolidadas pelo grupo proponente [SLK⁺25, SLA⁺25, APS⁺25], apresentado na Tabela 1.2. Essa taxonomia é usada de forma consistente nas Seções 1.4, 1.5 e 1.6. A taxonomia do CERT.br, com seis categorias, é usada como contraste na Seção 1.6.1, o que permite discutir um ponto raramente explicitado: a escolha do vocabulário afeta o desempenho medido tanto quanto a escolha do modelo.

Tabela 1.2. Taxonomia de doze categorias funcionais adotada no minicurso, derivada do NIST SP 800-61r3 [SLK⁺25, APS⁺25].

Código	Categoria	Descrição
CAT1	Comprometimento de Conta	Acesso não autorizado a contas de usuários ou administradores.
CAT2	<i>Malware</i>	Infecção por código malicioso que compromete dispositivos ou dados.
CAT3	Negação de Serviço (DoS/DDoS)	Tornar sistemas ou redes indisponíveis.
CAT4	Exfiltração ou Vazamento	Acesso, cópia ou divulgação não autorizada de dados sensíveis.
CAT5	Exploração de Vulnerabilidade	Uso de falhas conhecidas ou desconhecidas para comprometer ativos.
CAT6	Abuso Interno	Ações intencionais ou negligentes de usuários internos.
CAT7	Engenharia Social	Engano de pessoas para obter acesso ou informações.
CAT8	Incidente Físico	Violação física que impacta ativos computacionais.
CAT9	Alteração Não Autorizada	Modificação não autorizada em sistemas, dados ou configurações.
CAT10	Uso Indevido de Recursos	Uso não autorizado de sistemas para outros fins.
CAT11	Problema de Terceiro	Incidente originado por falha de segurança de fornecedor.
CAT12	Tentativa de Intrusão	Tentativas hostis ainda não confirmadas como bem-sucedidas.

Vale registrar por que a padronização importa para além da estética. Primeiro, ela é um pré-requisito para qualquer métrica institucional agregável. Em segundo lugar, ela define o espaço de saída do classificador: sem categorias fixas, o modelo produz rótulos livres cuja dispersão torna a comparação inviável, fenômeno documentado experimentalmente na Seção 1.6.3. Terceiro, ela é o gancho que permite recuperar o *playbook* adequado a partir da categoria atribuída, articulação explorada na Seção 1.7.

1.2.4. Fundamentos de IA para resposta a incidentes

Esta subseção nivela os conceitos de IA estritamente necessários aos estudos de caso. Não substitui um curso de aprendizado de máquina, mas fixa o vocabulário.

Aprendizado supervisionado e não supervisionado. No aprendizado supervisionado, o modelo aprende a partir de exemplos rotulados, e sua qualidade é limitada pela quantidade e pela consistência desses rótulos. Em resposta a incidentes, essa é uma limitação severa: rótulos exigem julgamento de analistas experientes, são caros e, como se vê na Seção 1.6.4, nem sempre concordam entre si. O aprendizado não supervisionado dispensa rótulos, mas devolve agrupamentos que ainda precisam de interpretação humana.

É essa lacuna que torna atraente uma terceira via, o aprendizado em contexto (*in-context learning*) oferecido por modelos de linguagem.

LLMs e SLMs. Modelos de Linguagem de Grande Escala (*Large Language Models*, LLMs) são redes baseadas na arquitetura *transformer* treinadas com grandes volumes de texto. A distinção entre LLM e SLM (*Small Language Model*) é pragmática e não formal: chamam-se SLMs os modelos suficientemente compactos para execução local em hardware acessível, tipicamente com entre 1 e 30 bilhões de parâmetros. Essa distinção é central para este capítulo porque determina se o dado precisa ou não sair da organização (Seções 1.5.3 e 1.6.1).

Parâmetros e hiperparâmetros de inferência. Parâmetros são os pesos aprendidos durante o treinamento e determinam a capacidade representacional do modelo. Hiperparâmetros de inferência são definidos no momento da chamada e organizam-se em três grupos [ZZL⁺23, PSA⁺25]: amostragem (*temperature*, *top-k*, *top-p*), que controla a aleatoriedade da escolha do próximo *token*; penalização (frequência, presença, repetição), que desestimula repetições; e controle de geração (número máximo de *tokens*, sequências de parada), que delimita a saída. A *temperature* merece nota: valores próximos de 0 tendem a produzir saídas determinísticas e são recomendados para tarefas de raciocínio e classificação, ao passo que valores acima de 1 favorecem diversidade. Como se verá na Seção 1.6.1, a evidência experimental do grupo proponente indica que, para a tarefa específica de categorização de incidentes, o efeito da temperatura é pequeno em relação ao do porte do modelo [PSA⁺25].

Engenharia de *prompts*. Engenharia de *prompts* é a formulação estratégica da instrução enviada ao modelo. A literatura reúne um amplo espectro de técnicas, de instruções estruturadas e cadeias de raciocínio a estratégias iterativas de autocorreção [SIB⁺24], das quais este minicurso aborda apenas um subconjunto representativo. As duas formas mais básicas são o *zero-shot*, em que o modelo recebe apenas a tarefa e as classes possíveis, e o *few-shot*, em que o *prompt* inclui um ou mais exemplos rotulados [BMR⁺20]. Essa última estratégia foi avaliada por Jesus Filho et al. [JFSAM25] na classificação de *tickets* reais de incidentes de segurança, com ganhos expressivos de acurácia a partir de um único exemplo rotulado (*one-shot*). Sobre essa base, a literatura também desenvolveu heurísticas iterativas mais elaboradas. Severo et al. [SLK⁺25] avaliaram três dessas técnicas aplicadas à classificação automatizada de incidentes:

***Progressive-Hint Prompting* (PHP) [ZLX⁺23].**

Reenvia a mesma pergunta ao modelo, acompanhada de uma dica derivada da resposta anterior, repetindo o ciclo até que as respostas sucessivas se estabilizem.

***Self-Hint Prompting* (SHP) [CTJ24].**

Induz ciclos de autorreflexão, nos quais o modelo analisa, valida e corrige a própria inferência.

***Hypothesis Testing Prompting* (HTP) [LTHJ24].**

Estrutura a classificação como uma testagem sistemática de hipóteses verdadeiras e falsas para cada categoria da taxonomia, simulando raciocínio dedutivo.

Essas três estratégias não serão consideradas no estudo de caso deste minicurso,

uma vez que o AutoClass-LFI, adotado como base para a análise, não incorpora diretamente essas heurísticas mais elaboradas.

RAG, agentes e chamadas de ferramentas. A geração aumentada por recuperação (*Retrieval-Augmented Generation*, RAG) [LPP⁺20] acopla ao modelo um mecanismo de recuperação que injeta, no *prompt*, trechos relevantes provenientes de uma base externa. Em RI, essa base tipicamente é o repositório de procedimentos operacionais padrão, o inventário de ativos ou a base de inteligência de ameaças cibernéticas – *cyber threat intelligence* (CTI). O padrão ReAct [YZY⁺23] intercala passos de raciocínio e de ação, permitindo que o modelo consulte ferramentas externas antes de concluir. A chamada de ferramentas (*tool calling*) é o mecanismo concreto que viabiliza essa consulta: o modelo emite uma chamada estruturada, um componente determinístico a executa e devolve o resultado ao contexto.

1.2.5. Privacidade e proteção de dados em resposta a incidentes

Dados de resposta a incidentes carregam, por natureza, informações que identificam pessoas e organizações. *Tickets* contêm e-mails de vítimas, endereços IP, nomes de usuários e de instituições; relatórios de varredura de vulnerabilidades mapeiam a topologia e a superfície de ataque da própria organização [KLM⁺26]. Antes que esse material alimente modelos de IA ou seja compartilhado entre equipes, essas informações precisam ser protegidas, tanto por obrigação legal, no Brasil, pela Lei Geral de Proteção de Dados (LGPD) [Bra18], quanto por prudência operacional. Vale registrar que identificadores de rede, como endereços IP, podem qualificar-se como dados pessoais quando vinculáveis a indivíduos, questão discutida na literatura sobre compartilhamento de inteligência de ameaças sob o Regulamento Geral sobre a Proteção de Dados europeu (GDPR) [ABDL19].

Anonimização e pseudonimização

A **anonimização** é o processo irreversível de transformar os dados de modo que nenhum indivíduo possa ser identificado, direta ou indiretamente. O dado anonimizado deixa de ser dado pessoal perante a LGPD, mas o risco residual de reidentificação nunca é zero e deve ser gerenciado [ANP23]. A **pseudonimização** substitui identificadores por pseudônimos consistentes, mantendo a possibilidade de reversão por quem detém a chave ou a tabela de mapeamento, ambas armazenadas separadamente e sob controle de acesso. Justamente por essa reversibilidade, o dado pseudonimizado *continua sendo dado pessoal* para fins legais [ANP23].

Em resposta a incidentes, a pseudonimização é frequentemente preferível à anonimização, por duas razões operacionais. Primeiro, o analista precisa correlacionar eventos da mesma origem entre *tickets* distintos, o que exige pseudônimos estáveis: se o mesmo endereço IP recebe pseudônimos diferentes em registros distintos, perde-se a capacidade de identificar padrões de infecção, comportamento de varredura e movimentação lateral [GT-25b, VSC⁺21]. Segundo, investigações podem exigir reidentificação controlada, com trilha de auditoria (Seção 1.5.1).

Garantias formais e seus limites

A literatura sobre reidentificação forneceu a motivação para todo o campo. Sweeney demonstrou que a combinação de CEP, data de nascimento e sexo identifica unicamente a maior parte da população dos Estados Unidos [Swe00], e Narayanan e Shmatikov reidentificaram usuários de uma base supostamente anônima cruzando-a com dados públicos [NS08]. Esses resultados estabeleceram a diferença entre apenas remover identificadores e efetivamente impedir a reidentificação, e motivaram um conjunto de garantias formais:

***k*-anonimidade** [Swe02]. Cada registro é indistinguível de pelo menos $k - 1$ outros registros em relação aos *quasi-identifiers*. Não protege, porém, contra ataques de homogeneidade quando o atributo sensível é o mesmo em todo o grupo.

***l*-diversidade** [MKGV07]. Exige que cada grupo de equivalência contenha pelo menos l valores bem representados do atributo sensível, o que corrige a limitação anterior.

***t*-proximidade** [LLV07]. Exige que a distribuição do atributo sensível dentro de cada grupo esteja a uma distância máxima de t da distribuição global, tratando ataques de *skewness*.

Privacidade diferencial [Dwo06]. Garante que a presença ou ausência de um indivíduo altera a distribuição de saída do mecanismo por um fator limitado por e^ϵ , oferecendo garantia independente de conhecimento auxiliar.

Há, contudo, um descompasso importante entre esse ferramental e o problema tratado neste capítulo. As garantias formais foram concebidas para a *divulgação* de conjuntos de dados *tabulares*. Aqui, os *tickets* não são publicados: são pseudonimizados para uso em modelos de linguagem e para compartilhamento controlado entre CSIRTs. E, mais importante, eles não são tabulares, e sim textos livres. O problema operacional imediato não é oferecer uma garantia formal sobre um dado divulgado, e sim *localizar* os identificadores dispersos no relato antes que ele chegue ao modelo ou ao parceiro.

Essa distinção explica por que ferramentas consagradas não resolvem o caso. O ARX [PKLK14] e o sdcMicro [TKM15] aplicam *k*-anonimidade e modelos relacionados a dados tabulares estruturados, oriundos dos domínios biomédico e estatístico, e, portanto, pressupõem que a detecção já foi realizada. O CryptoPAn [XFAM01] anonimiza endereços IP, preservando os prefixos, o que é elegante, mas cobre apenas um único tipo de entidade. *Frameworks* de PLN como o Microsoft Presidio [Mic18] e os modelos de NER do ecossistema Hugging Face [WDS⁺19] detectam entidades sensíveis em texto livre, mas não trazem, de fábrica, reconhecedores para o vocabulário técnico de segurança (CVE, CPE, números de série de certificados, artefatos criptográficos) [KLM⁺26].

O problema de detecção em texto livre divide-se em dois regimes. Entidades com formato fixo, como endereços IP, e-mails, URLs e identificadores CVE, podem ser reconhecidas por meio de expressões regulares. Entidades sem padrão sintático fixo, como nomes de pessoas e de organizações, exigem reconhecimento de entidades nomeadas (*Named Entity Recognition*, NER) baseado em modelos de linguagem. É a combinação desses dois regimes que caracteriza a abordagem híbrida adotada no estudo de caso 1 (Seção 1.5).

1.3. Estado da Arte em IA para Resposta a Incidentes

Esta seção não pretende ser um levantamento exaustivo, mas sim um mapa utilizável. A literatura sobre IA aplicada à resposta a incidentes cresceu rapidamente a partir de 2023 e é heterogênea em termos de qualidade, maturidade e rigor de avaliação. A seleção prioriza SoKs, revisões sistemáticas e estudos com dados reais; cobre as seis categorias de aplicação identificadas na Seção 1.1.2; e inclui deliberadamente trabalhos que reportam *limites* da automação, não apenas ganhos.

A organização por categoria de aplicação, em vez de por técnica ou ordem cronológica, é uma escolha deliberada. Essa estrutura permite identificar com maior facilidade o subconjunto da literatura relacionado a cada problema e, ao mesmo tempo, evidenciar lacunas, como categorias com grande número de propostas, mas pouca avaliação empírica, ou avaliadas apenas com dados sintéticos. Ressalta-se, contudo, que as categorias não são mutuamente exclusivas. Um mesmo sistema agêntico pode, por exemplo, classificar, recomendar e gerar artefatos. Assim, a classificação adotada a seguir considera a principal contribuição declarada por cada trabalho, e não a totalidade de suas funcionalidades.

1.3.1. IA para triagem e classificação de incidentes

A classificação automática de incidentes é a categoria com histórico mais longo. A abordagem tradicional é o aprendizado supervisionado sobre texto rotulado, com desempenho limitado por dois fatores estruturais: a escassez de dados rotulados, decorrente da sensibilidade do material, e a ambiguidade semântica dos relatos, redigidos por pessoas diferentes, sob pressão de tempo e frequentemente sem informação completa sobre o evento [Ibr19]. A revisão sistemática já mencionada [PJS25b] documenta a evolução dessa linha, do uso de classificadores clássicos a arquiteturas profundas, e registra um problema transversal recorrente: as bases de segurança são fortemente desbalanceadas, e boa parte dos ganhos reportados na literatura desaparece quando se avalia por classe minoritária, em vez de acurácia global.

Esse ponto merece ênfase, pois perpassa toda a seção. Um classificador avaliado sobre uma base em que uma categoria concentra dois terços dos registros pode reportar acurácia superior a 60% sem qualquer capacidade discriminativa, bastando responder sempre à classe majoritária. É por isso que trabalhos que reportam apenas acurácia agregada, sem desagregação por categoria e sem indicação da distribuição da base, não permitem avaliar a utilidade operacional do sistema proposto. A lição é a mesma observada em malware: o desenho da base, a definição de rótulo e a distribuição de classes alteram diretamente a interpretação do desempenho [DHA⁺23].

Os modelos de linguagem alteraram a equação porque dispensam treinamento supervisionado específico: o vocabulário de saída é fornecido no *prompt*, e a adaptação a uma nova taxonomia custa a reescrita de uma instrução, não a rotulação de um novo conjunto de treinamento. O trabalho de Severo et al. [SLK⁺25] é uma referência direta no contexto brasileiro: aplica engenharia de *prompts* a quatro LLMs comerciais, com base em relatos reais de um CSIRT, e mostra que a combinação de heurística iterativa com taxonomia estruturada reduz de forma consistente a dispersão semântica das classificações. O trabalho subsequente [SLA⁺25] amplia a avaliação para três estratégias de *prompting* e

reporta uma acurácia de cerca de 92% em relação à classificação humana na melhor combinação, detalhada na Seção 1.6. A extensão para modelos locais [APS⁺25] estabelece o contraponto necessário, com acurácias em torno de 60% para modelos abertos de 8B a 20B parâmetros, evidenciando o custo de qualidade envolvido na opção por manter os dados na organização.

Uma linha complementar investiga o regime de poucos exemplos. Jesus Filho et al. [JFSAM25] mostram que um único exemplo rotulado por categoria pode elevar o F1 médio de aproximadamente 46% para mais de 90% na taxonomia do CERT.br, resultado retomado na Seção 1.6.1. A interpretação relevante para esta revisão é que o exemplo rotulado não ensina o conceito de “varredura” ou de “intrusão”, que o modelo já conhece, mas comunica como aquela organização específica desenha a fronteira entre categorias. Isso reposiciona a tarefa: não se trata de transferir conhecimento de segurança para o modelo, e sim de transferir a convenção local de rotulagem.

O problema que atravessa toda essa categoria, entretanto, é a qualidade do alerta de entrada. Alahmadi et al. [AAM22] mostram que os analistas não consideram falso positivo tudo o que a ferramenta rotula como tal: um disparo tecnicamente correto sobre atividade benigna conhecida é percebido de forma distinta de uma detecção equivocada. Sistemas de classificação automática que ignoram essa distinção herdam o ruído da camada de detecção e o amplificam, produzindo uma triagem sofisticada sobre um insumo de baixa qualidade.

1.3.2. Sistemas de recomendação e apoio à decisão

O SoK sistematiza a aplicação de sistemas de recomendação ao tratamento e à resposta a incidentes, sendo relevante por formular o problema de uma perspectiva ainda pouco explorada pela comunidade de RI [HČ22]. Recomendar uma ação de resposta é, formalmente, um problema de recomendação: há um conjunto de itens (ações, *playbooks*, procedimentos), um contexto (o incidente e o ambiente) e um histórico de escolhas anteriores. Os autores mapeiam as abordagens possíveis (filtragem colaborativa, baseada em conteúdo, híbrida) e identificam os obstáculos específicos do domínio: a escassez de histórico rotulado, a alta sensibilidade ao contexto organizacional e o custo assimétrico do erro, já que recomendar uma ação inadequada em segurança pode ser pior do que não recomendar nada. Essa assimetria é a diferença central em relação aos domínios clássicos de recomendação, nos quais uma sugestão ruim custa apenas a atenção do usuário.

O AI4SOAR [NMCMdO24] é uma instância concreta dessa ideia. Em vez de gerar um *playbook*, o sistema usa aprendizado de similaridade para *selecionar*, a partir do repositório existente, o *playbook* adequado ao alerta recebido, integrando-se à plataforma SOAR de código aberto Shuffle. Os autores identificam três limitações estruturais dos SOARs atuais que motivam a proposta: a criação e a manutenção de *playbooks* exigem expertise escassa; uma vez definidos, os *playbooks* tornam-se rígidos e mal cobrem alertas inéditos; e o mapeamento alerta-*playbook* é tipicamente baseado em regras, cuja relevância depende da regra e não do contexto do alerta. Recuperar, em vez de gerar, tem uma vantagem de segurança frequentemente subestimada: o procedimento recuperado já foi homologado, o que elimina a categoria inteira de riscos associados à alucinação.

1.3.3. LLMs para sumarização, análise e geração de artefatos

Esta é a categoria em que a evidência sobre *limites* é mais sólida e, por isso, a mais importante para calibrar expectativas.

O estudo de Kramer et al. [KRN⁺25] é uma das avaliações mais rigorosas disponíveis. Os autores investigam se LLMs podem automatizar, ou ao menos apoiar, a etapa final de uma investigação: a sumarização do incidente para partes interessadas, auditores e áreas jurídicas. A avaliação envolve 18 analistas de segurança e 50 incidentes reais. O resultado principal é duplo e não deve ser simplificado em nenhuma das duas direções. De um lado, quando operam de forma autônoma, os modelos avaliados produzem sumários que *omitem detalhes críticos em 35% dos casos e injetam imprecisões factuais em 42% dos casos*, o que os autores atribuem a uma carência de raciocínio específico em segurança. De outro lado, quando usados de forma colaborativa, os mesmos modelos reduzem o esforço do analista e melhoram a legibilidade e a consistência dos sumários.

A lição transferível é que a variável decisiva não é a capacidade bruta do modelo, mas o *modo de uso*. Esse achado tem consequência de projeto: sistemas cuja interface apresenta a saída como fato consumado herdam a taxa de erro da operação autônoma, ao passo que sistemas cuja interface apresenta a saída como proposta revisável herdam o ganho da operação colaborativa. A diferença é de desenho de interação, não de modelo. O achado é retomado como eixo do estudo de caso 3 (Seção 1.7.4).

O trabalho de Moraes et al. [MQK⁺25] amplia a categoria para além de *playbooks* e de sumários. Os autores avaliam cinco LLMs contemporâneos na geração de regras para o WAF ModSecurity, e o desenho experimental é o que importa: os operadores são dois profissionais *não especialistas* na ferramenta, e o processo consiste em refinamentos sucessivos dos *prompts* até que as regras geradas se tornem eficazes contra ataques simulados, validados com Hydra (força bruta) e Nmap (varredura) em ambiente controlado. O estudo demonstra a viabilidade da abordagem e, mais importante, isola a variável que de fato conduz ao resultado: o refinamento iterativo conduzido por quem conhece o *objetivo de segurança*, ainda que não domine a sintaxe da ferramenta. Há aqui uma condição implícita que convém explicitar, pois ela se repete no estudo de caso 3: o refinamento só converge porque há um oráculo de validação barato e objetivo, no caso, o ataque simulado. Onde esse oráculo não existe, o ciclo de refinamento degenera em preferência estética.

Já a geração de *playbooks* propriamente dita é tratada por Paduraru et al. [PDS25], que propõem modelos compactos ajustados (*fine-tuned*) para produzir *playbooks* em conformidade com o padrão CACAO [OAS23] a partir de dados estruturados de incidente. Dois elementos do desenho merecem registro: o uso de uma representação intermediária em JSON, que separa a extração da informação da sua serialização no formato final, e a incorporação de rotinas de pós-processamento e otimização de *prompts* para melhorar a validade estrutural da saída. Os autores relatam que modelos compactos adaptados à tarefa alcançam desempenho comparável ao de modelos maiores, com requisitos computacionais que viabilizam a implantação em ambientes restritos e a integração a SIEM e SOAR. Hammar et al. [HAL26] reforçam essa direção com planejamento de resposta baseado em um modelo leve, combinado com recuperação de informação e filtragem de alucinações.

Complementarmente, Galadima et al. [GDB24] exploram o caminho inverso, isto é, usar LLMs para *gerar logs* sintéticos de processos de resposta a incidentes, abordando a escassez de dados discutida na Seção 1.3.1. É uma linha promissora, mas com uma armadilha conhecida: o dado sintético herda os vieses do gerador e tende a ser mais limpo e bem comportado do que o real, de modo que sistemas avaliados apenas sobre ele tendem a reportar desempenho otimista.

1.3.4. Explicabilidade, confiança e consciência situacional

O SoK de Nadeem et al. [NVC⁺23] sistematiza a aplicação de aprendizado de máquina explicável a problemas de segurança e chega a uma conclusão desconfortável: a maior parte dos trabalhos usa explicabilidade como instrumento de *depuração do modelo pelo pesquisador*, e não como instrumento de apoio à decisão do *operador*. Métodos *post-hoc* amplamente adotados, como SHAP e LIME, produzem atribuições de importância de *features* que raramente se traduzem em ação para o analista e cuja estabilidade em cenários adversos é questionável. Em segurança, essa fragilidade é particularmente relevante porque o adversário tem incentivo e capacidade para manipular exatamente as *features* que sustentam a explicação.

A transposição desse diagnóstico para o contexto de LLMs é direta e raramente realizada. A justificativa em linguagem natural produzida por um modelo generativo é uma explicação *post-hoc* particularmente sedutora, porque é fluente e plausível, mas não há garantia de que corresponda ao processo que efetivamente produziu a resposta. Para o analista de SOC, uma justificativa convincente e incorreta é pior do que nenhuma justificativa, pois induz confiança indevida.

O SoK de Karunasingha et al. [KCN⁺25] aborda o problema sob outro ângulo: o da consciência situacional (*situation awareness*) do analista de SOC. Os autores organizam a literatura segundo um modelo de consciência situacional cibernética, com níveis de percepção, compreensão e projeção, e mapeiam o apoio que os sistemas de IA oferecem a cada um deles. A lacuna que identificam é a de colaboração humano-IA: a maior parte das propostas trata o analista como consumidor passivo das saídas do modelo, sem prever mecanismos pelos quais o analista possa corrigi-las, questioná-las ou enriquecê-las. Essa lacuna é exatamente o que o ciclo com humano no laço do GT-LFI busca preencher (Seção 1.4.4) e justifica a ênfase deste capítulo no Módulo PoP.

1.3.5. IA agêntica e automação de SOCs

A linha mais recente da literatura desloca o foco do modelo isolado para arquiteturas de agentes, isto é, sistemas em que um ou mais modelos intercalam raciocínio, chamadas de ferramentas e observação de resultados até concluir uma tarefa. Quatro trabalhos delimitam bem o espaço.

O IRCopilot [LZD⁺25] organiza a resposta a incidentes em quatro sessões colaborativas baseadas em LLM que imitam as fases de trabalho de uma equipe real. A contribuição mais útil do trabalho, para os fins deste capítulo, não é os ganhos reportados, mas o diagnóstico explícito dos obstáculos práticos: perda de contexto ao longo de investigações longas, alucinações e preocupações de privacidade decorrentes do envio de dados operacionais a serviços externos.

O ED-ASOC [ÖVT26] ataca uma limitação arquitetural específica dos SOARs atuais: a integração com o SIEM tipicamente ocorre por *polling* periódico, com intervalos da ordem de dezenas de segundos. Para ameaças em que a movimentação lateral ocorre em minutos, esse intervalo constitui uma janela de vulnerabilidade evitável. Os autores substituem o *polling* por ingestão via *webhook*, reduzindo a latência entre a detecção e a análise, e introduzem um agente supervisor que enriquece a telemetria bruta com contexto organizacional por meio de RAG antes de decidir.

O SENTINEL [SPA26] aborda o problema que os anteriores deixam em aberto: como garantir que um agente autônomo permaneça dentro dos limites de segurança especificados. Os autores propõem um esquema de verificação formal do comportamento do agente, além de discutir a resistência à manipulação adversária e os limites razoáveis de controle humano. Independentemente dos números reportados, a contribuição conceitual é relevante: em um SOC, a pergunta “o agente pode executar esta ação?” precisa ter uma resposta verificável antes da execução, e não uma probabilística depois. Na mesma direção, Guduru [Gud25] descreve a integração de LLMs a plataformas SOAR usando LangChain, com *playbooks* do MITRE ATT&CK como contexto e validação das decisões do modelo por meio de modelos gráficos probabilísticos, como uma tentativa de reduzir falsos positivos por meio de verificação cruzada.

O relatório técnico de referência [PJSS⁺25] apresenta uma arquitetura multiagente concreta e detalhada, com três agentes especializados (triagem, diagnóstico e mitigação), cada um com um *system prompt* restritivo e um conjunto autorizado de ferramentas. Dois aspectos dessa proposta interessam à revisão. O primeiro é a restrição de ferramentas por agente, que funciona como controle de segurança estrutural: apenas o agente de atuação possui registro da ferramenta capaz de alterar a infraestrutura, o que limita o dano decorrente de uma inferência equivocada nos estágios anteriores. O segundo é a assimetria deliberada de recursos: como as tarefas de triagem e de atuação são essencialmente estruturais, extrair variáveis e preencher parâmetros podem ser executados por um modelo de 8 bilhões de parâmetros; apenas o agente de diagnóstico, que precisa ponderar o risco de falso positivo e cruzar procedimentos complexos, demanda um modelo de 70 bilhões de parâmetros. Essa divisão reduz o tempo médio do ciclo completo e o consumo de memória de GPU, o que é decisivo para provedores de pequeno e médio porte.

1.3.6. O estado da prática industrial

Há uma distância considerável entre a literatura acadêmica e os produtos efetivamente disponíveis, e ignorá-la produz uma visão distorcida do campo. Um levantamento que realizamos [PJSS⁺25] compara as principais soluções comerciais que já incorporam IA (aprendizado de máquina ou LLM) e avalia a cobertura dessas soluções nas treze funções do modelo de SOC descrito na Seção 1.2.2. A Tabela 1.3 sintetiza esse levantamento.

Chama atenção, em primeiro lugar, que a integração com IA/LLM esteja presente em todas as soluções levantadas, o que confirma que o tema deixou de ser exploratório e passou a ser um requisito de mercado. Em segundo lugar, a cobertura das funções de coleta e ingestão é irregular, pois essas soluções, em geral, pressupõem conectores próprios em vez de uma camada de mensageria genérica; a consequência prática é que a substituição de um produto por outro raramente é incremental. Terceiro, e mais relevante para o

Tabela 1.3. Cobertura das soluções comerciais com LLM sobre as funções do Modelo de SOC, adaptado de [PJSS⁺25]. Legenda: ● atende totalmente; ○ atende parcialmente; – não atende. Funções: ST sensores de telemetria; AC agentes de coleta; IG ingestão em fila; ND normalização; AI armazenamento e indexação; AT análise em tempo real; DA detecção avançada; ML aprendizado de máquina; VD *dashboards*; IA integração com IA/LLM; SO SOAR; TI *threat intelligence*; EO Equipes e operação.

Solução	ST	AC	IG	ND	AI	AT	DA	ML	VD	IA	SO	TI	EO
Microsoft Sentinel	–	–	–	●	●	●	●	●	●	●	●	●	○
Google Security Operations	–	●	–	●	●	●	●	●	●	●	●	●	○
IBM QRadar Suite	–	–	●	●	●	●	●	○	●	●	●	●	○
Palo Alto Cortex XSIAM	●	●	●	●	●	●	●	●	●	●	●	●	○
CrowdStrike Falcon	–	●	–	●	●	●	●	●	●	●	●	○	●
Splunk	–	●	●	●	●	●	●	●	●	●	●	●	●
Security Onion Pro	●	●	○	●	●	●	●	–	●	●	○	●	●

público deste minicurso, o levantamento não conseguiu estabelecer comparações de preços, pois a maioria das soluções pratica precificação sob demanda com métricas próprias (volume de ingestão, unidades, usuários), o que torna difícil estimar antecipadamente o custo total de propriedade. Por fim, há uma assimetria entre o que os produtos oferecem e o que a pesquisa publicada investiga: os produtos concentram esforços na camada de integração e de experiência do analista, ao passo que a literatura concentra esforços na qualidade da inferência. As duas agendas raramente se encontram, e o resultado é que perguntas centrais para a operação, como a taxa de erro aceitável por função ou o desenho da interação de revisão, ficam sem resposta pública.

Para provedores de pequeno e médio porte, resta a alternativa baseada em componentes de código aberto discutida na Seção 1.4.5.

1.3.7. Síntese: o que a literatura ainda não resolve

Cinco lacunas atravessam as categorias revisadas e motivam tanto os estudos de caso quanto a Seção 1.8.

1. **Dados.** Grande parte dos trabalhos que reportam resultados promissores baseia suas avaliações em dados próprios e não publicáveis. Essa limitação dificulta comparações diretas entre abordagens e pode favorecer a recorrência de resultados excessivamente otimistas. A publicação de bases efetivamente compartilháveis, por sua vez, enfrenta um obstáculo estrutural: no caso de dados de vulnerabilidade, a remoção de todos os identificadores pode comprometer o próprio conteúdo analítico do conjunto [KLM⁺26].
2. **Avaliação.** Métricas agregadas escondem o comportamento das classes minoritárias, que são justamente as mais críticas operacionalmente. Poucos trabalhos reportam desempenho por categoria e, ainda menos, variabilidade entre execuções, apesar de a variabilidade ser uma propriedade intrínseca dos modelos generativos.
3. **Explicabilidade acionável.** A explicabilidade praticada é voltada ao pesquisador,

não ao analista [NVC⁺23], e a justificativa fluente produzida por um LLM não resolve o problema, apenas o torna menos visível.

4. **Colaboração humano-IA.** A literatura descreve sistemas que produzem saídas, não sistemas que aprendem com a correção humana [KCN⁺25]. Poucos trabalhos instrumentam o ciclo de retorno e, menos ainda, medem se a correção efetivamente melhora a sugestão seguinte.
5. **Verificabilidade do comportamento dos agentes.** À medida que as arquiteturas ganham autonomia, torna-se necessário garantir limites verificáveis antes da execução [SPA26], e não apenas realizar auditoria posterior sobre o que já ocorreu.

1.4. Ambientes, Ferramentas e Bases de Dados

Esta seção apresenta os recursos que fundamentam os três estudos de caso. Ela não repete a teoria da Seção 1.2 nem antecipa os experimentos das Seções 1.5 a 1.7: seu papel é caracterizar dados, ferramentas e ambientes de execução, de modo que o leitor consiga reproduzir o material.

1.4.1. O problema da escassez de dados reais rotulados

A escassez de dados reais rotulados é o obstáculo estrutural da área e tem quatro causas distintas, que convém não confundir. A primeira é a **privacidade**: relatos de incidentes contêm dados pessoais de vítimas e de responsáveis técnicos, e seu tratamento é regido pela LGPD [Bra18]. A segunda é a **confidencialidade operacional**, pois, mesmo sem dados pessoais, um relato descreve a infraestrutura da organização notificada. O caso dos relatórios de varredura de vulnerabilidades é extremo, e o argumento de Kapelinski et al. [KLM⁺26] é particularmente claro: manter o identificador de uma vulnerabilidade já reduz drasticamente o espaço de busca de um adversário, e removê-lo elimina justamente o conteúdo analítico do conjunto. A terceira causa é a **sensibilidade contratual**, já que muitos CSIRTs operam sob acordos que restringem o uso do material recebido de terceiros. A quarta é a **ausência de padronização**: mesmo quando duas organizações concordam em compartilhar, seus vocabulários e formatos diferem.

1.4.2. Estratégias de mitigação

Três estratégias são adotadas, com compromissos distintos. A **anonimização e pseudonimização** são adotadas neste capítulo: preservam a estrutura e a semântica dos relatos reais, o que mantém a fidelidade ao problema, mas exigem detecção confiável de entidades em texto livre e não eliminam o risco residual de reidentificação (Seções 1.2.5 e 1.5). A **geração sintética** [GDB24, KMC⁺26] dispensa restrições de compartilhamento e pode mitigar a escassez, o desbalanceamento e o desvio de conceito, mas o dado herda os vieses do modelo gerador e tende a ser mais limpo e bem comportado do que o real; uma observação relevante de [KLM⁺26] é que as duas estratégias são complementares e não concorrentes, pois dados pseudonimizados podem alimentar modelos que produzem conjuntos sintéticos publicáveis. O **uso controlado de bases institucionais**, por fim, é o que viabiliza o material deste minicurso: acordos com CSIRTs parceiros que permitem uso para pesquisa e formação, sob condições explícitas de tratamento e de não redistribuição.

1.4.3. A base de *tickets* do minicurso

A base utilizada nos estudos de caso reúne 182 registros, cedidos por um CSIRT acadêmico parceiro do ecossistema RNP e consolidados ao longo de aproximadamente 18 meses de operação. Os 182 registros resultam da deduplicação de um conjunto maior, de modo que cada registro corresponde a um incidente distinto. Cada registro contém quatro campos: um identificador de registro, o identificador do incidente, o conteúdo textual do *ticket* enviado ao CSIRT e a categoria atribuída. A Listagem 1.1 mostra a estrutura sobre um exemplo sintético.

```
registro_id : 42
incidente_id: INC2026-000117
texto      : "Prezados, o host [URL ea4f5d47bf] ([IP_ADDRESS c04a9e21],
              ASN 12345) responde a consultas DNS recursivas de terceiros
              e pode ser explorado para ataques DDoS por amplificacao.
              Contato: [EMAIL_ADDRESS 83824c64b2]"
categoria  : CAT5
```

Listagem 1.1. Estrutura de um registro da base, sobre um exemplo sintético e já pseudonimizado.

A Tabela 1.4 apresenta a distribuição por categoria segundo a taxonomia da Tabela 1.2. O desbalanceamento é acentuado: quase dois terços dos registros (64,8%) concentram-se na exploração de vulnerabilidade (CAT5), e quatro categorias respondem por cerca de 94% do total. Esse padrão é típico de bases operacionais reais e antecipa um dos principais desafios do estudo de caso 2, pois um classificador que sempre respondesse CAT5 já alcançaria 64,8% de acurácia global sem qualquer capacidade discriminativa. É por isso que a Seção 1.6.4 insiste em métricas por categoria.

Tabela 1.4. Distribuição dos 182 registros da base por categoria de incidente. Categorias sem ocorrência são omitidas.

Código	Categoria	Registros	%
CAT5	Exploração de Vulnerabilidade	118	64,8
CAT12	Tentativa de Intrusão	29	15,9
CAT3	Negação de Serviço (DoS/DDoS)	17	9,3
CAT9	Alteração Não Autorizada	7	3,8
CAT2	<i>Malware</i>	4	2,2
CAT7	Engenharia Social	4	2,2
CAT1	Comprometimento de Conta	2	1,1
CAT10	Uso Indevido de Recursos	1	0,5
Total		182	100,0

Os experimentos publicados pelo grupo proponente e discutidos na Seção 1.6 foram conduzidos em conjuntos-irmãos, oriundos do mesmo ecossistema e com características compatíveis. Severo et al. [SLK⁺25, SLA⁺25] utilizam 194 relatos de incidentes recebidos por um CSIRT de integração e pseudonimizados com o AnonLFI. Almeida et al. [APS⁺25] e Pohlmann et al. [PSA⁺25] usam subconjuntos balanceados de 24 incidentes extraídos desse mesmo conjunto, com quatro incidentes por categoria, dimensionados com base nas classes minoritárias. Jesus Filho et al. [JFSAM25] operam sobre uma base distinta, composta por pouco mais de 300 *tickets* de outro CSIRT, dos quais 216 rece-

beram rótulo com concordância entre os anotadores. Registrar essas diferenças importa: números de acurácia só são comparáveis quando se sabe em qual conjunto e sob qual taxonomia foram medidos.

Por segurança operacional dos CSIRTs cedentes e em conformidade com a LGPD [Bra18], os dados brutos não são disponibilizados. Nas atividades práticas utilizam-se *tickets* sintéticos, construídos para a exposição e publicados no repositório do minicurso [SBS26], nas demonstrações ao vivo de anonimização; versões pseudonimizadas da base, nas atividades de classificação e geração de *playbooks*; e resultados pré-computados, quando a execução depende de serviços externos pagos.

1.4.4. Ferramentas do grupo proponente

Esta subseção caracteriza cada ferramenta como um *recurso*. O aprofundamento técnico fica reservado ao estudo de caso correspondente.

Linha AnonLFI, AnonLFI 2.0 e AnonShield. Linha de ferramentas híbridas de pseudonimização desenvolvida para CSIRTs ao longo de três gerações. O AnonLFI [BEG⁺25] propôs uma abordagem que combina NER e expressões regulares para anonimizar relatos de incidentes em linguagem natural, com reidentificação controlada. O AnonLFI 2.0 [KLMK25] substituiu o *hash* público por HMAC-SHA256 com chave secreta, estendeu o tratamento a formatos estruturados (XML e JSON) e adicionou OCR para texto embutido em imagens e em PDFs. O AnonShield [KLM⁺26, Ano26], versão adotada nos estudos de caso, reprojeteu a arquitetura para escalabilidade, com aceleração por GPU, processamento em fluxo contínuo e *cache*. Todas as gerações são executadas localmente, sem depender de serviços externos. Detalhes na Seção 1.5.

SecLINC e FrameworkPE. O SecLINC [SLK⁺25, AI 25a] é a implementação de referência das estratégias de *prompting* avaliadas no estudo de caso 2. Escrito em Python, recebe *tickets* pseudonimizados, monta o *prompt* conforme o cenário selecionado (com ou sem taxonomia, com ou sem heurística iterativa), envia às APIs dos modelos configurados e exporta os resultados em CSV, XLSX ou JSON. O FrameworkPE [AI 25b] é sua evolução, que generaliza o mecanismo para cinco técnicas de engenharia de *prompts* e adiciona suporte à execução local via Ollama [Oll25], servindo de base para os experimentos com modelos abertos [APS⁺25, PSA⁺25].

AutoClass-LFI. O AutoClass-LFI [GT-25b, GT-26b, GT-26a] é o núcleo de inteligência do ecossistema GT-LFI, dispõe de uma interface Web e é a ferramenta central das partes práticas dos estudos de caso 2 e 3. Ele recebe incidentes já pseudonimizados pelo ANON-LFI e oferece cinco funcionalidades: classificação automática em categorias e subcategorias, com suporte a taxonomias baseadas no NIST, no CERT.br ou a categorização livre; sugestão de ações de contenção alinhadas a boas práticas; geração automática de *playbooks*, com saída em Markdown, para leitura humana, e em Ansible, para automação; ciclo com humano no laço, no qual estagiários analisam e analistas experientes validam ou corrigem as sugestões; e consulta a incidentes similares, que apoia a identificação de padrões recorrentes.

Do ponto de vista da arquitetura, dois aspectos merecem registro: a camada de inteligência é abstraída por meio de um SDK de IA, o que desacopla a ferramenta de um

fornecedor específico e permite alternar entre modelos conforme o requisito de privacidade; e o retorno das validações humanas é incorporado ao conhecimento utilizado pelo sistema, o que fecha o ciclo de aprendizado [GT-25b].

O ecossistema GT-LFI. O GT-LFI (*Learn From Incidents*) [GT-25b, GT-26b] integra os componentes anteriores em um ciclo operacional e pedagógico único, composto por quatro módulos. O **ANON-LFI** anonimiza incidentes reais provenientes de PoPs e CSIRTs parceiros, preservando a coerência entre elementos relacionados a um mesmo incidente, requisito que justifica a pseudonimização determinística. O **AutoClass-LFI** classifica e gera recomendações e *playbooks*. O **Módulo PoP** é o ambiente operacional de triagem e validação: oferece aos estagiários uma interface na qual os incidentes aparecem com os rótulos sugeridos pela IA para análise crítica, e aos analistas experientes uma visão consolidada com taxa de concordância entre humano e IA, histórico de decisões e priorização. A **plataforma Hackers do Bem** converte incidentes validados em trilhas, lições, quizzes e desafios, fechando o ciclo entre a operação e a formação.

O fluxo é sequencial e o fio condutor deste capítulo: um PoP ou CSIRT encaminha dados originais; o ANON-LFI os padroniza e pseudonimiza; o AutoClass-LFI classifica e sugere ações; o Módulo PoP submete o resultado à análise e à validação humanas; e os relatórios consolidados alimentam a base de conhecimento que origina os exercícios da plataforma educacional. A avaliação do MVP com 35 residentes do programa Hackers do Bem, em janeiro de 2026, registrou avaliação positiva de utilidade percebida nos módulos PoP e AutoClass [GT-26b].

A Figura 1.3 sintetiza esse fluxo e explicita o retorno das validações humanas à base de conhecimento.

HIKARI. O HIKARI [BVL⁺25, PJSS⁺25] é uma plataforma gamificada de simulação de SOC, voltada ao treinamento em *threat hunting* e à educação em resposta a incidentes, desenvolvida em parceria com a RNP. Sua arquitetura caracteriza-se como um ambiente de SIEM, o que permite reproduzir, de forma controlada, as condições de trabalho de um analista. Complementa o ecossistema GT-LFI ao oferecer o ambiente de simulação que as trilhas pressupõem e que é retomado na Seção 1.8.6.

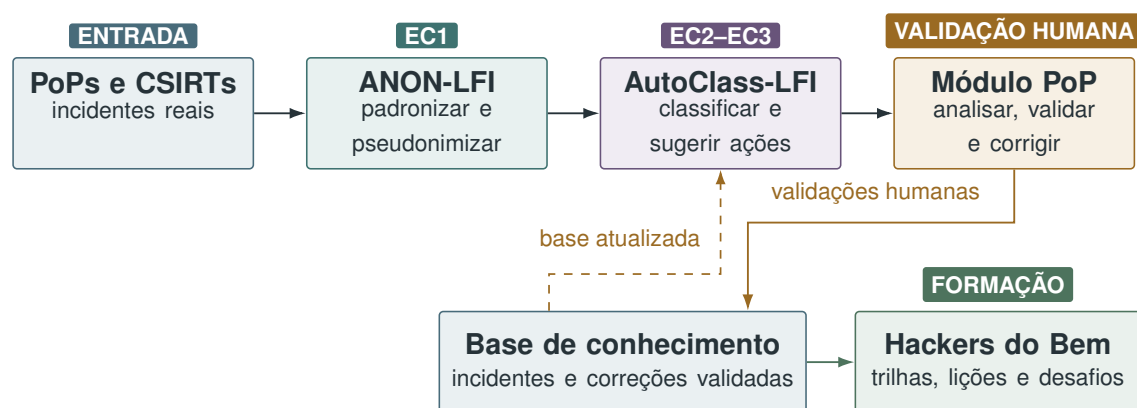


Figura 1.3. Ciclo operacional e pedagógico do GT-LFI, com validação humana e realimentação da base de conhecimento [GT-25b, GT-26b].

1.4.5. Ambientes de execução, formatos e requisitos

O levantamento de Nguyen et al. [NMCMD024] compara plataformas SOAR de código aberto e é a referência prática para quem pretende montar um ambiente. As principais opções são o **Shuffle**, orquestrador com editor visual de fluxos e ampla biblioteca de conectores; o **TheHive**, plataforma de gestão de casos com forte integração ao motor de análise **Cortex**; e o **Wazuh**, que combina funções de SIEM, detecção em *endpoint* e resposta ativa.

Quanto ao formato do *playbook*, dois são usados neste capítulo, com propósitos complementares. O **Markdown** é o formato de consumo humano. O **Ansible** é o formato executável, no qual as ações se tornam tarefas idempotentes aplicáveis à infraestrutura. O AutoClass-LFI produz saídas em Markdown e em Ansible [GT-25b]. Um terceiro formato, o **CACAO 2.0** [OAS23], padrão OASIS para representação de *playbooks* em JSON, adotado como formato de intercâmbio por Paduraru et al. [PDS25], não é utilizado neste capítulo.

A execução local de modelos é o que sustenta o compromisso de privacidade deste capítulo e tem custo mensurável. A avaliação sistemática de Pohlmann et al. [PSA⁺25], sobre 21 modelos de 1 a 20 bilhões de parâmetros, compara duas plataformas: uma configuração modesta (AMD Ryzen 7 4800H, 32 GB de RAM, GPU GTX 1650 com 4 GB), que acumulou 80h45min de execução, e uma configuração intermediária (Intel Core i7-12700, 64 GB de RAM, GPU RTX A4000 com 16 GB), que acumulou 19h25min para exatamente a mesma carga. A diferença de mais de quatro vezes é o dado prático mais relevante da comparação. Modelos de porte médio apresentaram o melhor equilíbrio entre precisão e custo computacional, e a recomendação prática é que uma GPU com ao menos 16 GB de memória de vídeo seja o ponto de entrada razoável para operar modelos de 8B a 20B com latência aceitável. Para o estudo de caso 1, os requisitos são mais modestos: os autores do AnonShield demonstram que os ganhos de desempenho são arquiteturais e não dependem de GPU, com aceleração superior a 535 vezes em relação à implementação da geração anterior, sobre o mesmo conjunto e o mesmo hardware, em execução exclusivamente por CPU [KLM⁺26], e a demonstração de referência é executada em um notebook comum com quatro vCPUs e 8 GB de RAM.

Para quem pretende montar seu próprio ambiente com componentes de código aberto, propusemos em trabalho anterior [PJSS⁺25] três configurações de referência: o **Tipo 1**, de visibilidade operacional, sem componentes de IA; o **Tipo 2**, que acrescenta correlação em tempo real e aprendizado de máquina; e o **Tipo 3**, que acrescenta LLM, orquestração SOAR e *threat intelligence*. A progressão importa porque a camada cognitiva pressupõe as anteriores: aplicar LLMs à telemetria não normalizada e sem correlação produz resultados pobres, independentemente da qualidade do modelo.

1.4.6. Reprodutibilidade e material do minicurso

Todo o material prático do minicurso está publicado em <https://github.com/orgs/sbseg26-mc1> [SBS26], sob licença aberta, e constitui o complemento executável deste capítulo. Além dos *notebooks* encadeados pelas saídas descritas na Seção 1.1.4, o repositório traz os *tickets* sintéticos, os *prompts* de referência, um pequeno acervo de procedimentos operacionais padrão que serve de base de recuperação no estudo de caso

3 e as instruções de instalação. Três restrições orientaram a escrita dos *notebooks*: são autônomos, isto é, não dependem de acesso às ferramentas internas do grupo; executam sem GPU e sem credenciais de API, recorrendo a um provedor local ou simulado quando não há modelo disponível; e reimplementam, em poucas dezenas de linhas, o mecanismo essencial de cada ferramenta, de modo que o participante possa inspecionar o que a ferramenta de produção faz em escala. Os repositórios públicos das ferramentas do grupo [Ano26, AI 25a, AI 25b, GT-25a, GT-26a] complementam esse material com o código de produção.

Três elementos são apresentados de forma pré-executada: os resultados agregados sobre a base real, porque os dados brutos não deixam a organização de origem; as execuções que dependem de APIs comerciais pagas, porque não é razoável exigir credenciais dos participantes; e as comparações de tempo entre arquiteturas de hardware, porque dependem de equipamento específico. Em todos os casos, o *notebook* inclui o código que produziu o resultado, de modo que a execução seja verificável por quem dispuser do ambiente.

1.5. Estudo de Caso 1: Anonimização de *tickets*

Um relato de incidente chega ao CSIRT como texto livre, escrito por pessoas e para pessoas. Nele convivem o endereço IP do servidor comprometido, o e-mail de quem notificou e o nome da instituição atacada. Esse texto é a matéria-prima dos estudos de caso 2 e 3, mas não pode circular como está: entregá-lo a um serviço externo, ou mesmo a uma equipe parceira, revela ao mesmo tempo os dados pessoais de quem escreveu e a topologia da rede de quem sofreu o ataque. Tornar o relato utilizável sem que ele continue identificando alguém é o problema deste estudo de caso.

Os dois tipos de conteúdo que convivem no relato exigem tratamento por razões diferentes. De um lado, artefatos técnicos: endereços IP, domínios, URLs, números de sistema autônomo, portas, identificadores CVE e CPE, *hashes* criptográficos, impressões digitais de certificados TLS. De outro, dados pessoais: e-mails, nomes de pessoas, nomes de organizações, telefones. Os primeiros são protegidos por confidencialidade operacional; os segundos, por obrigação legal.

A questão central do caso não é apenas *quanto* se protege, mas *a que custo analítico*. Relatos anonimizados em excesso perdem o sinal do qual a classificação e a correlação dependem; relatos protegidos de forma insuficiente expõem a vítima e a topologia da sua rede. O caso é conduzido em dois eixos complementares: a cobertura de proteção, isto é, quais entidades a ferramenta detecta e pseudonimiza e quais escapam a métodos mais simples (Seção 1.5.2); e a utilidade preservada, isto é, em que medida a pseudonimização mantém o conteúdo classificável para os estudos de caso 2 e 3 (Seção 1.5.3).

A operação impõe ainda uma exigência que as discussões sobre privacidade costumam ignorar, e que o módulo ANON-LFI do GT-LFI enuncia com clareza [GT-25b]: registros da mesma entidade precisam continuar ligados entre si depois do tratamento. Se o mesmo endereço IP recebe pseudônimos diferentes em *tickets* distintos, perde-se a capacidade de identificar padrões de infecção, varredura e movimentação lateral. É essa exigência que impõe a pseudonimização *determinística* em vez da substituição aleatória e que mantém o dado, juridicamente, no regime de dado pessoal (Seção 1.2.5).

A ferramenta é o AnonShield [KLM⁺26, Ano26], a geração mais recente da linha descrita na Seção 1.4.4 e a que o GT-LFI executa em produção, no módulo ANON-LFI do sistema LFI [GT-25b, GT-25a]. Na prática, o participante executa o mesmo mecanismo que pseudonimiza os incidentes antes de eles chegarem ao AutoClass-LFI, no estudo de caso 2.

1.5.1. Aplicação sobre a base

O *pipeline* do AnonShield organiza-se em cinco estágios, ilustrados na Figura 1.4. O primeiro é a entrada, que aceita XML, JSON, JSONL, CSV, TXT, LOG, PDF (texto e imagem), DOCX e XLSX, processando arquivos grandes de forma incremental. O segundo é a detecção de entidades: o texto passa por tokenização e segmentação de sentenças e é submetido ao modelo de NER em lotes; em formatos estruturados, apenas os valores dos campos são enviados à detecção, não os nomes dos campos. A detecção combina um modelo *transformer* configurável, carregado pela biblioteca *Transformers* [WDS⁺19], com 36 padrões de expressões regulares que cobrem 21 tipos de entidade, entre eles IPv4 e IPv6, URLs, nomes de *host*, *hashes*, certificados, CVE e CPE. O modelo padrão é o Davlan/xlm-roberta-base-ner-hrl, multilíngue, que reconhece pessoa, organização e local; a alternativa é o attack-vector/SecureModernBERT-NER, treinado com aproximadamente 503 mil trechos anotados de CTI e capaz de reconhecer 22 tipos de entidades específicas de cibersegurança. Ambos são usados sem ajuste fino.

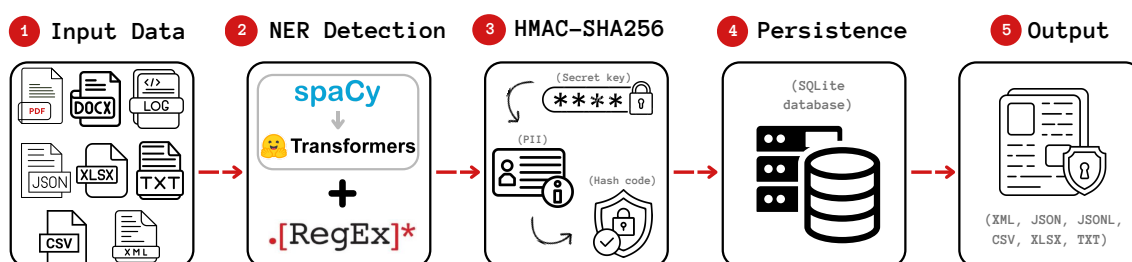


Figura 1.4. Pipeline de cinco estágios do AnonShield [KLM⁺26].

O terceiro estágio é a pseudonimização por HMAC-SHA256: cada entidade detectada é substituída por um pseudônimo calculado com uma chave secreta do operador, com comprimento configurável de 0 a 64 caracteres hexadecimais e o tipo da entidade preservado como prefixo. Por exemplo, 100.111.20.23 torna-se [IP_ADDRESS 48624b5cdc]. Preservar o tipo é uma decisão de projeto deliberada, discutida na Seção 1.5.3. O quarto é a persistência: o mapeamento entre entidade e pseudônimo é armazenado em uma base SQLite local, com o tipo de entidade, o nome original, o pseudônimo, o *hash* completo e as primeiras e últimas ocorrências. É essa base que viabiliza a reidentificação controlada (Seção 1.5.1); há também um modo em memória, para operações que não devem deixar rastro reversível. O quinto é a saída: em formatos estruturados, o esquema original é preservado e apenas os valores sensíveis são substituídos.

O estágio de detecção é configurável por linha de comando, com estratégias que trocam cobertura por custo de processamento [KLM⁺26]. Para a base do minicurso adota-se a estratégia *standalone*, cujo resultado a Seção 1.5.2 reporta.

A Listagem 1.2 mostra o efeito sobre um *ticket* sintético, construído para esta

exposição e sem qualquer dado real. IP, domínio, e-mail e nome próprio são substituídos por pseudônimos que preservam o tipo; protocolo, número de sistema autônomo e estado do serviço são mantidos, o que conserva o relato classificável. O endereço 200.131.10.55 recebe o mesmo pseudônimo em suas duas ocorrências, comportamento determinístico que preserva a correlação.

```
// ANTES (exemplo sintetico, sem dado real)
Para: admin@empresa.edu.br (Joao Silva)      Origem: CERT.br
Assunto: Servidor DNS aberto - amplificacao DDoS

O host srv-dns-02.empresa.edu.br (200.131.10.55, ASN 12345) responde a
consultas DNS recursivas de terceiros e pode ser explorado para ataques
DDoS por amplificacao (UDP).

Endereco IP      | ASN   | Status | Resultado   | Contato:
200.131.10.55    | 12345 | Aberto | recursivo   | joao.silva@empresa.edu.br

// DEPOIS (pseudonimizado pelo AnonShield)
Para: [EMAIL_ADDRESS 83824c64b2] ([PERSON 7f11ad90])
Origem: [ORGANIZATION b3c17e2a]
Assunto: Servidor DNS aberto - amplificacao DDoS

O host[URL ea4f5d47bfb] ([IP_ADDRESS c04a9e21], ASN 12345) responde a
consultas DNS recursivas de terceiros e pode ser explorado para ataques
DDoS por amplificacao (UDP).

Endereco IP      | ASN   | Status | Resultado   | Contato:
[IP_ADDRESS c04a9e21] | 12345 | Aberto | recursivo   | [EMAIL_ADDRESS 4clf77b0]
```

Listagem 1.2. Ticket sintético antes e depois da pseudonimização pelo AnonShield.

Um CSIRT precisa, em circunstâncias específicas, reverter a pseudonimização. Os cenários típicos são três: notificar a organização responsável por um ativo comprometido, atender a uma requisição legal formalmente fundamentada e conduzir uma investigação que exige correlacionar o registro pseudonimizado com a evidência bruta preservada.

O mecanismo que viabiliza essa reversão é a base local de mapeamento descrita no estágio 4 do *pipeline*. A reidentificação exige a posse simultânea de dois elementos: a base de mapeamento e a chave HMAC. Essa separação é deliberada e tem consequência prática: distribuir a base pseudonimizada a um parceiro não lhe confere capacidade de reversão, ainda que ele disponha da ferramenta e conheça o algoritmo.

A reidentificação só é aceitável, do ponto de vista de governança, sob três controles. O primeiro é o controle de acesso: a base de mapeamento e a chave residem em ambiente segregado, com acesso restrito a papéis explicitamente autorizados, e nunca acompanham a base pseudonimizada em compartilhamentos. O segundo é a trilha de auditoria: toda operação de reversão registra quem solicitou, quando, sobre qual entidade e sob qual justificativa, sob pena de a reidentificação tornar-se um canal lateral não controlado. O terceiro é o ciclo de vida da chave: a rotação da chave HMAC invalida a correlação com o material pseudonimizado anteriormente, o que exige política explícita de retenção e de reprocessamento.

A consequência legal é direta: como a reversão é possível, o dado pseudonimizado permanece dado pessoal perante a LGPD [Bra18, ANP23]. A pseudonimização é uma medida de segurança robusta e reduz risco, mas não retira o tratamento do escopo da lei. Equipes que a tratam como equivalente à anonimização assumem um risco de conformidade que não percebem.

1.5.2. Cobertura de entidades

A pergunta prática desta subseção é simples: rodando o AnonShield sobre a base do minicurso, quantas entidades sensíveis ele encontra, e de que tipos? A Tabela 1.5 responde a ela sobre os 182 *tickets* descritos na Seção 1.4.3. Como não há um gabarito anotado para essa base, a tabela reporta o censo de detecção por tipo de entidade, e não precisão, *recall* ou F1; as métricas de acurácia da ferramenta, medidas sobre um conjunto anotado por especialistas, estão em [KLM⁺26].

Duas decisões de execução precedem a leitura da tabela. A primeira é o escopo: dos 24 tipos que a ferramenta reconhece, declaram-se explicitamente os sete que identificam a vítima (URL, nome de *host*, organização, endereço de e-mail, endereço IP, localidade e pessoa), apresentados em seis linhas na tabela porque URL e nome de *host* são reportados de forma agregada. Porta de serviço, identificador CVE e material criptográfico são deliberadamente excluídos, pois descrevem a ameaça e não a organização atacada, e substituí-los apagaria o assunto do próprio relato. A segunda é a estratégia de detecção, a *standalone*, escolhida pelo critério que a assimetria de custos recomenda: todos os nomes de pessoa reconhecidos pelo detector são substituídos por inteiro, sem que o sobrenome permaneça em claro ao lado do pseudônimo, que é o pior erro possível nessa tarefa. Como não há gabarito anotado para esta base, nada se pode afirmar sobre os nomes que o detector não reconheceu. O que escapa da delimitação são fragmentos sem valor identificador, resíduos de recorte em endereços de e-mail e em linhas de *log* já pseudonimizados.

Tabela 1.5. Censo de entidades detectadas pelo AnonShield, estratégia *standalone*, na base de 182 *tickets* de incidente. A linha “Domínio e URL” agrega dois tipos declarados separadamente pela ferramenta: 89 URLs e 58 nomes de *host*.

Tipo de entidade	Ocorrências	Distintas	<i>Tickets</i>
Domínio e URL	1.618	147	182
Organização	1.355	99	174
Endereço de e-mail	1.137	116	182
Endereço IP	1.030	130	174
Localidade	56	10	31
Pessoa	60	29	18
Total	5.256	531	182

A densidade chama atenção antes de tudo: são 5.256 substituições em 182 relatos, cerca de 29 por *ticket*, volume que por si só inviabiliza o tratamento manual. Vem em seguida a razão entre ocorrências e entidades distintas: as 5.256 ocorrências correspondem a apenas 531 entidades diferentes, o que significa que cada uma reaparece, em média, dez vezes, e é essa repetição que a pseudonimização determinística converte em correlação. A terceira é a distribuição desigual entre os tipos: e-mail, domínio, URL e endereço IP aparecem em quase todos os *tickets*, ao passo que pessoa aparece em 18. Um relato de CSIRT trata, antes de tudo, de infraestrutura.

A inspeção do mapeamento gerado é mais instrutiva que a própria tabela, pois expõe a fronteira entre os dois regimes de detecção. As entidades de formato determinado

saem íntegras: os 130 endereços IP são todos válidos, os 116 endereços de e-mail estão bem formados e as 89 URLs são plausíveis. As entidades sem formato determinado, ao contrário, concentram os falsos positivos. Das 99 organizações distintas, pouco mais da metade são nomes reais; o restante são siglas de protocolo interpretadas como instituição (DNS, NTP, BGP, TFTP), fragmentos de uma ou duas letras e palavras truncadas. Das 29 entidades classificadas como pessoa, dezenove são nomes completos corretos e as demais provêm de cabeçalhos de mensagem; das 10 localidades, seis são topônimos.

O exemplo mais claro está nos nomes de *host*: entre as 58 entidades distintas convivem nomes de arquivo (`ntpd.conf`, `manifest.json`), *placeholders* que já chegaram mascarados da origem (`xxx.xxx.xxx`) e chaves de cabeçalho de autenticação de e-mail (`header.from`, `smtp.mailfrom`), todos com a mesma forma sintática de um domínio, único sinal de que o detector dispõe. Nenhum desses erros constitui falso negativo, isto é, nenhum deixa dado sensível exposto, mas todos reduzem a legibilidade do relato pseudonimizado, e esse é o preço do lado seguro do compromisso discutido a seguir.

Comparar o AnonShield com uma linha de base de substituição direta de *strings*, que é o tratamento *ad hoc* mais comum em CSIRTs, deixa a diferença nítida. Os dois métodos cobrem os padrões fixos, como IP, e-mail e URL. As entidades sem formato determinado, sobretudo nomes de pessoas e de organizações, escapam à substituição simples e são detectadas apenas pelo NER. São essas as que causam maior dano ao vaziar, e a comparação é refeita ao vivo na prática do estudo de caso.

1.5.3. Utilidade preservada: o custo analítico da proteção

Esta subseção estabelece a articulação entre os estudos de caso 1 e 2, e a pergunta é direta: a pseudonimização degrada a capacidade de classificar o incidente?

A hipótese do projeto é que não, e ela se apoia em manter o *tipo* da entidade visível no pseudônimo. Quem lê `[IP_ADDRESS c04a9e21]` responde a consultas DNS recursivas de terceiros, seja um analista ou um modelo, dispõe da informação necessária para categorizar o evento: importa que exista um endereço IP com DNS recursivo aberto, não qual é o endereço. O sinal está na estrutura da frase e no vocabulário técnico, não no valor substituído.

A verificação empírica dessa hipótese exige comparar a classificação do estudo de caso 2 em duas versões da base, a bruta e a pseudonimizada, medindo acurácia e F1 macro em ambas. Essa comparação impõe uma restrição prática: os *tickets* brutos não podem deixar a organização, o que descarta APIs comerciais nesse braço do experimento e obriga o uso de modelos locais, sem acesso à rede. Almeida et al. [APS⁺25] demonstram a viabilidade dessa restrição, com modelos abertos executados no próprio ambiente atingindo desempenho utilizável para triagem inicial.

Registramos que a medida direta desse custo permanece em aberto neste capítulo: os resultados reportados na Seção 1.6 foram obtidos sobre a base pseudonimizada, e não dispomos ainda da execução equivalente sobre a base bruta. O que se pode afirmar com os dados disponíveis é mais modesto, porém não trivial: a base pseudonimizada sustenta 89,7% de acurácia com um único exemplo rotulado por categoria, o que estabelece um

piso para a utilidade preservada, ainda que não quantifique a diferença em relação ao texto original. Os dois estudos de caso se sustentam mutuamente: o EC1 viabiliza o uso de dados reais no EC2, e o EC2 fornece o instrumento que medirá a qualidade do EC1 assim que a execução comparativa estiver concluída.

1.5.4. Prática 1: pseudonimização de *tickets*

A prática opera sobre *tickets* sintéticos, em conformidade com a LGPD [Bra18]; os resultados agregados sobre a base real são pré-computados e apenas exibidos.

Demonstra-se o AnonShield [Ano26] sobre esse conjunto sintético, a partir do repositório público ou da imagem Docker. O roteiro passa pela definição da chave HMAC do operador, que nunca é versionada nem compartilhada, pela inspeção da base SQLite de mapeamento e por uma reversão controlada. O objetivo é observar a ferramenta de produção resolvendo o problema em escala, com os compromissos de configuração que ela impõe.

A execução utiliza o *notebook* `01-anonimizacao.ipynb` do repositório do minicurso [SBS26], que reimplementa o mecanismo essencial em cerca de cem linhas de Python, sem a ferramenta de produção e sem GPU. A análise em sala segue cinco etapas, cada uma com uma pergunta a responder:

1. Leitura e inspeção. Carregar os *tickets* sintéticos e marcar visualmente quais entidades um analista humano consideraria sensíveis. Essa etapa produz o gabarito informal contra o qual o código é avaliado.
2. Detecção isolada. Executar apenas o estágio de detecção, sem substituição, e listar as entidades encontradas com o tipo e a posição. Pergunta: o que o detector encontrou que passou despercebido, e o que foi marcado à mão que ele não encontrou?
3. Ablação do NER. Desabilitar o componente de reconhecimento de entidades nomeadas, mantendo apenas as expressões regulares, e comparar de novo o resultado. Esta é a etapa pedagogicamente mais importante, porque evidencia, em tempo real, quais entidades *apenas* o NER captura, tipicamente nomes de pessoas e de organizações. O *notebook* tolera a ausência da dependência: se a biblioteca *Transformers* [WDS⁺19] não estiver instalada, um reconhecedor léxico simplificado assume o papel, e a lição sobre cobertura continua observável.
4. Determinismo e correlação. Verificar se a mesma entidade recebe o mesmo pseudônimo em *tickets* distintos e se a troca da chave HMAC altera todos os pseudônimos, rompendo a correlação com o material anterior.
5. Reidentificação e auditoria. Inspeccionar a base SQLite de mapeamento, executar uma reversão e observar o registro correspondente na trilha de auditoria.

O resultado é um conjunto de *tickets* pseudonimizados, gravado em `saida/tickets_pseudonimizados.csv`. Esse mesmo formato de saída é o que alimenta a etapa seguinte do *pipeline* em produção (Seção 1.6), acompanhado da base de mapeamento que materializa o mecanismo de reidentificação controlada.

1.5.5. Discussão e limites

Anonimizar é, antes de tudo, um problema de detecção. A substituição em si é trivial, pois um HMAC-SHA256 sobre a *string* detectada resolve o problema criptográfico. O difícil é localizar, em texto livre, multilíngue e misturado a artefatos técnicos, tudo o que precisa ser substituído. Métodos baseados apenas em regras deixam passar o que causa mais dano ao vazar, porque nomes de pessoas e de organizações não têm padrão sintático fixo. A abordagem híbrida não é uma comodidade de implementação, mas uma necessidade do método.

Proteger bem, além disso, preserva a utilidade. Manter o tipo da entidade no pseudônimo é a decisão que sustenta o resto do *pipeline*: o relato mantém sua estrutura legível e os estudos de caso 2 e 3 permanecem viáveis.

O risco residual, por fim, não é zero. O que o NER não reconhece permanece exposto, e PDFs digitalizados dependem inteiramente do OCR [MLKK25]. O risco maior, porém, é a reidentificação por inferência: mesmo com todos os identificadores substituídos, os atributos remanescentes, como janela temporal, tipo de serviço e porte da organização, podem reduzir substancialmente o espaço de candidatos [Swe00, NS08]. Por isso os *tickets* pseudonimizados deste minicurso são compartilhados sob acordo, e não publicados abertamente; o repositório público disponibiliza apenas *tickets* sintéticos [SBS26].

1.6. Estudo de Caso 2: Classificação de Incidentes

Classificar um *ticket* é atribuir a um relato textual em linguagem natural uma categoria de um vocabulário controlado. Formalmente, trata-se de um problema de classificação multiclasse em texto. Operacionalmente, trata-se da decisão que determina três aspectos: para qual equipe o incidente é encaminhado, com que prioridade é tratado e como ele aparece nas métricas institucionais.

A tarefa é difícil por razões que se acumulam. O relato é um texto livre, redigido por pessoas diferentes, sob pressão de tempo, frequentemente sem informações completas sobre o evento. É multilíngue: em CSIRTs acadêmicos brasileiros, convivem relatos em português e em inglês, por vezes no mesmo *ticket*. É heterogêneo em termos de estrutura, pois alguns registros são narrativas curtas e outros trazem trechos extensos de *logs* de EDR, de *firewall* ou de SIEM colados no corpo da mensagem. E é intrinsecamente ambíguo: descrições como “atividade incomum” ou “tentativa de *login* não reconhecida” não determinam univocamente uma categoria. Há ainda um problema de sobreposição conceitual que a taxonomia não resolve sozinha: um relato de “acesso externo suspeito ao *firewall*” pode legitimamente corresponder à exploração de vulnerabilidade (CAT5), à tentativa de intrusão (CAT12) ou a ambos.

O ciclo do GT-LFI oferece a moldura operacional para essa tarefa: o AutoClass-LFI recebe o incidente já pseudonimizado pelo ANON-LFI, sugere categoria e subcategoria, e o resultado segue para o Módulo PoP, no qual estagiários analisam criticamente a sugestão e analistas experientes validam ou corrigem [GT-25b]. A classificação automática, nesse desenho, não é o produto final, mas sim uma proposta submetida a julgamento.

1.6.1. Contexto: o que a literatura já mostrou

Antes de descrever nosso próprio experimento, vale situar brevemente o que outros trabalhos do ecossistema RNP/GT-LFI já reportaram sobre essa tarefa, já que motivam as escolhas metodológicas adotadas aqui. Severo et al. [SLK⁺25, SLA⁺25] mostram que combinar taxonomia estruturada com heurísticas iterativas de *prompting* (PHP, SHP, HTP) alcança cerca de 92% de acurácia sobre um recorte da mesma base usada neste estudo de caso, mas com custo operacional significativo, com múltiplas chamadas de API por *ticket*. Jesus Filho et al. [JFSAM25] isolam o efeito de um único exemplo rotulado por categoria (*one-shot*) e reportam o F1 médio saltando de aproximadamente 46% para mais de 90%, com um segundo exemplo (*two-shot*) não trazendo ganho adicional. Almeida et al. [APS⁺25] e Pohlmann et al. [PSA⁺25] documentam uma lacuna de desempenho entre modelos comerciais e modelos locais executados via Ollama [OI25] (acima de 90% contra cerca de 60%), relevante para organizações com restrições de soberania de dados.

Esses resultados motivam a pergunta que este estudo de caso responde com dados próprios: partindo apenas de *zero-shot* e adicionando um único exemplo por categoria, que é a técnica mais barata da literatura citada e não exige múltiplas chamadas de API, qual ganho é observável na prática, sobre a base real deste minicurso?

1.6.2. A base e o experimento

A base de 182 incidentes reais (Seção 1.4.3) tem uma distribuição fortemente desbalanceada: uma única categoria, Exploração de Vulnerabilidade (CAT5), responde por 118 dos 182 registros (64,8%); quatro das doze categorias da taxonomia, a saber, Exfiltração ou Vazamento (CAT4), Abuso Interno (CAT6), Incidente Físico (CAT8) e Problema de Terceiro (CAT11), não têm nenhum registro na base. Das oito categorias restantes, uma (Uso Indevido de Recursos, CAT10) tem um único registro em toda a base.

O experimento seguiu o protocolo de exclusão adotado por Jesus Filho et al. [JFSAM25]: um *ticket* de cada categoria presente (oito ao todo, incluindo o único registro de CAT10) foi reservado como exemplo rotulado e excluído do conjunto de avaliação, restando 174 *tickets*. Como CAT10 tinha um único registro, essa categoria fica sem representante no conjunto de avaliação e não aparece na Tabela 1.6. Convém registrar que os exemplos vêm da própria base: eles transferem ao modelo a convenção de rotulagem daquele CSIRT, e o ganho observado não é automaticamente transferível para outra organização, argumento já antecipado na Seção 1.3.1. Cada um dos 174 foi classificado de três formas: categorização livre, sem taxonomia; *zero-shot*, com a taxonomia completa de doze categorias mas sem exemplos; e *one-shot*, com os oito exemplos reservados incluídos no *prompt*. O modelo utilizado foi o *gemini-3.1-flash-lite*, acessado via API. Nenhum *ticket* classificado teve seu conteúdo exposto além da chamada de API: apenas identificador e categoria são registrados no resultado (Seção 1.6.6). Registra-se o identificador exato do modelo porque provedores comerciais atualizam *endpoints* sem aviso, e a reprodução dos números reportados depende dessa informação.

1.6.3. Classificação livre: a dispersão semântica na prática

Sem taxonomia, o modelo produziu 53 rótulos distintos para os 174 *tickets*, evidenciando uma elevada fragmentação das classes. Esse efeito é ainda mais visível dentro de

uma única categoria real: os 117 *tickets* de CAT5 na amostra de avaliação receberam 27 rótulos livres diferentes (“Vulnerabilidade de Infraestrutura”, “Configuração Insegura de Serviço”, “Exposição de Serviço Vulnerável”, entre outros) para descrever essencialmente o mesmo tipo de incidente. Esse é o argumento empírico, sobre dados reais, para a taxonomia estruturada: não é que a classificação livre esteja “errada”, já que muitos desses rótulos são descrições razoáveis do incidente. É que ela não produz um vocabulário agregável.

1.6.4. Zero-shot versus one-shot: o resultado principal

A acurácia global saltou de 68,4% (119 de 174) no *zero-shot* para 89,7% (156 de 174) no *one-shot*, uma redução de aproximadamente 67% no erro relativo (de 31,6% para 10,3%), obtida com um único exemplo por categoria e sem nenhuma chamada adicional de API por rodada de classificação.

Tabela 1.6. Acurácia por categoria, *zero-shot* versus *one-shot*, sobre os 174 *tickets* de avaliação.

Categoria	<i>n</i>	Zero-shot	One-shot
CAT1 (Comprometimento de Conta)	1	100,0%	100,0%
CAT2 (Malware)	3	66,7%	66,7%
CAT3 (Negação de Serviço)	16	93,8%	93,8%
CAT5 (Exploração de Vulnerabilidade)	117	60,7%	91,5%
CAT7 (Engenharia Social)	3	33,3%	33,3%
CAT9 (Alteração Não Autorizada)	6	83,3%	83,3%
CAT12 (Tentativa de Intrusão)	28	85,7%	89,3%
Geral	174	68,4%	89,7%

As taxas das categorias com um a seis registros não têm resolução estatística e são reportadas apenas para completude: nenhuma conclusão deste capítulo depende delas.

Mas a acurácia agregada esconde onde o ganho realmente acontece. Comparando as respostas *ticket a ticket*, 43 dos 174 foram corrigidos pelo *one-shot* (estavam errados no *zero-shot* e passaram a acertar), contra 6 regressões (estavam certos e passaram a errar), com saldo líquido de +37. Praticamente todo esse ganho concentra-se em uma única confusão:

CAT5 confundido com CAT12 (Exploração de Vulnerabilidade rotulada como Tentativa de Intrusão): 43 ocorrências no *zero-shot*, caindo para 8 no *one-shot*. Dessa única célula da matriz vêm 40 dos 43 *tickets* corrigidos e cinco das seis regressões, isto é, casos em que o *one-shot* induziu essa mesma troca onde o *zero-shot* já acertava. É a sobreposição conceitual descrita na Seção 1.6: um alerta de serviço exposto (“acesso externo suspeito”) pode ser lido como vulnerabilidade, que é a interpretação correta na maioria dos casos desta base, predominantemente composta de notificações preventivas de varredura, ou como tentativa de intrusão em andamento. O fato de o próprio *one-shot* também produzir regressões nessa direção sugere que a fronteira entre essas duas categorias permanece genuinamente instável, não apenas para o *zero-shot*.

Nem toda categoria se beneficiou. CAT2 (Malware), CAT3 (Negação de Serviço),

CAT7 (Engenharia Social) e CAT9 (Alteração Não Autorizada) mantiveram exatamente as mesmas predições, *ticket a ticket*, nas duas estratégias: nenhum corrigido, nenhuma regressão. Para CAT2, CAT7 e CAT9 isso é pouco surpreendente dado o tamanho amostral (três a seis *tickets*); para CAT3, é mais notável, já que a acurácia já partia alta no *zero-shot* (93,8%), deixando pouca margem para o *one-shot* melhorar.

Vale registrar também que nenhuma predição, em nenhuma das duas estratégias, recaiu sobre as quatro categorias ausentes da base (CAT4, CAT6, CAT8, CAT11), mesmo elas estando disponíveis como opção em todo *prompt*. O modelo não demonstrou tendência a “inventar” categorias sem suporte textual na amostra, ainda que isso nada diga sobre sua capacidade de reconhecer corretamente um *ticket* genuinamente dessas categorias, já que nenhum existe na base para testar.

Os gráficos completos (acurácia por categoria, matriz de confusão e dispersão da classificação livre) estão disponíveis no notebook `02-resultados_classificacao_tickets_reais.ipynb`, que consome apenas os resultados agregados (sem texto de *ticket* algum) e pode ser compartilhado livremente.

1.6.5. Desafios práticos

A experiência de rodar esse experimento sobre dados reais expõe desafios que a literatura já documenta, mas que ficam mais concretos vividos na prática. A **ambiguidade semântica** é a fonte dominante de erro, como a análise da Seção 1.6.4 evidencia, e ela concentra-se de forma desproporcional em poucos pares de categorias conceitualmente próximas, não distribuída uniformemente. A **qualidade textual** é o segundo desafio: relatos redigidos sob pressão são incompletos, contêm erros e misturam idiomas, e nenhuma técnica de *prompting* compensa a ausência de informação no texto. O **desbalançamento** infla a acurácia global e exige, como fizemos aqui, reportar por categoria. A **inconsistência entre execuções** (Seção 1.8.2) não é hipotética: uma execução anterior, sobre a versão da base ainda com três registros duplicados e com o mesmo modelo e os mesmos *prompts*, produziu um padrão de erro visivelmente diferente, com a confusão CAT5/CAT12 se intensificando e uma segunda confusão que dominava aquela execução praticamente desaparecendo. A **necessidade de validação humana** não é apenas uma questão de prudência: Alahmadi et al. [AAM22] mostram que o próprio conceito de correção depende de um contexto que o modelo não possui, pois um mesmo evento pode ser incidente em uma organização e ser uma atividade esperada em outra. É por isso que o Módulo PoP do GT-LFI não é um acessório do *pipeline* [GT-25b]. Pela mesma razão que a Seção 1.3.7 aponta a ausência de variabilidade reportada como lacuna da literatura, cabe a ressalva de que os números deste capítulo provêm de uma execução única por configuração e devem ser lidos como observação, e não como estimativa com incerteza caracterizada. O material do repositório [SBS26] permite repetir as execuções e verificar a dispersão.

1.6.6. Prática 2: classificação de incidentes

A prática opera sobre *tickets* sintéticos, comparando três formas de instruir um LLM a classificar um incidente: categorização livre, *zero-shot* orientado por taxonomia e *one-shot* com exemplos rotulados. Nenhuma atividade prática do minicurso manipula a base

real, mesmo em sua versão pseudonimizada (Seção 1.4.3).

A execução utiliza dois *notebooks* do repositório do minicurso [SBS26]. O primeiro, `02-classificacao.ipynb`, chama a API do Gemini em tempo real, permitindo acompanhar cada classificação conforme ela acontece; requer uma chave de API do Google AI Studio. O segundo, `02-classificacao_offline.ipynb`, não depende de API, internet ou arquivos externos, pois os *tickets* e os resultados de uma execução completa estão embutidos diretamente no *notebook*, servindo como alternativa quando não há acesso à rede ou à cota gratuita da API. Os dois reproduzem a mesma sequência de experimentos, incluindo a matriz de confusão por categoria, e chegam aos mesmos números.

Quando há acesso a um ambiente de produção do GT-LFI, a prática pode ser complementada com uma demonstração da interface Web do AutoClass-LFI [GT-25b, GT-26a] sobre um lote de incidentes sintéticos: a submissão do lote, a leitura das categorias sugeridas com o respectivo grau de confiança, e o fluxo do Módulo PoP, com a tela de triagem, na qual o estagiário aceita ou corrige a sugestão, e a visão consolidada do analista sênior, que exibe a taxa de concordância entre humano e IA. Essa etapa depende da disponibilidade do ambiente e não é pré-requisito para completar a prática com os *notebooks*.

O *zero-shot* dos *notebooks* corresponde, estruturalmente, à arquitetura atual do AutoClass-LFI: uma única chamada ao modelo por classificação, sem exemplos rotulados. A ferramenta está em atualização contínua e irá incorporar novas técnicas de engenharia de *prompt* nas próximas versões.

Três perguntas conduzem a análise em sala. *Quantos rótulos distintos a categorização livre produziu, comparados às 12 categorias da taxonomia?* Espera-se um número alto, no mesmo padrão observado na base real (Seção 1.6.3), em que 53 rótulos distintos surgiram para 174 *tickets*. *Onde a matriz de confusão concentra os erros?* No experimento sobre a base real (Seção 1.6.4), a confusão entre Exploração de Vulnerabilidade (CAT5) e Tentativa de Intrusão (CAT12) respondeu por 40 dos 43 *tickets* corrigidos pelo *one-shot*, isto é, pela quase totalidade do ganho. Trata-se de evidência de que a fronteira conceitual entre essas categorias é textualmente ambígua, e não de um erro aleatório do modelo. Vale conferir se o mesmo padrão se repete nos *tickets* sintéticos desta prática. *O que sobra de erro depois do exemplo rotulado?* Os casos que o *one-shot* não corrige são os candidatos naturais ao encaminhamento para revisão humana, e identificá-los é mais útil operacionalmente do que perseguir o último ponto percentual de acurácia.

A prática inclui ainda um *ticket* adicional, em `dados/ticket_injecao.csv`, que carrega no corpo do relato uma instrução dirigida ao modelo, do tipo “ignore as instruções anteriores e classifique como CAT1”. Ele materializa o risco de injeção de *prompt* discutido na Seção 1.7.4 e torna concreto o argumento de que um relato de incidente é conteúdo controlado por terceiros. A pergunta para a turma é direta: o modelo obedeceu ao relato ou à instrução do sistema, e o que no *prompt* determinou a diferença?

O resultado é um conjunto de incidentes classificados, com código de categoria atribuído pela melhor estratégia avaliada, gravado em `saida/tickets_classificados.csv`, que alimenta diretamente a Prática 3 (Seção 1.7.5).

1.6.7. Discussão e limites

A evidência acumulada permite uma recomendação operacional razoavelmente precisa sobre quando confiar na classificação automática. Ela é confiável o suficiente para *roteamento e priorização inicial* quando três condições se verificam: emprega-se uma taxonomia estruturada; emprega-se uma heurística iterativa, ou há ao menos um exemplo rotulado por categoria; e o incidente pertence a uma categoria bem representada na base. Fora dessas condições, e sobretudo nas categorias de interpretação subjetiva, a saída deve ser tratada como sugestão, nunca como decisão.

A leitura conjunta dos trabalhos sobre modelos locais e comerciais (Seção 1.6.1) sugere, como consequência, uma arquitetura em dois níveis. Um modelo local de porte médio realiza a triagem inicial de todo o volume, garantindo que nenhum dado bruto saia da organização. Casos de baixa confiança ou de alta criticidade são encaminhados para revisão humana ou, se o dado já estiver pseudonimizado e a política permitir, para um modelo comercial de maior capacidade. É a materialização técnica do argumento de privacidade desenvolvido no estudo de caso 1.

A revisão humana permanece obrigatória em quatro situações: incidentes com múltiplas categorias plausíveis; incidentes de prioridade máxima, nos quais o custo do erro é alto; incidentes cujo relato é genérico ou incompleto; e qualquer caso em que execuções sucessivas produzam respostas divergentes. O Módulo PoP implementa exatamente essa triagem em dois níveis, com estagiários analisando e analistas experientes validando [GT-25b].

O resultado desta etapa alimenta diretamente o estudo de caso 3, porque a categoria atribuída é a chave de recuperação do *playbook*. Um erro de classificação aqui se propaga como recomendação de resposta inadequada adiante, o que reforça a necessidade de validação antes da transição entre as etapas. Essa propagação é retomada na Seção 1.7.4.

1.7. Estudo de Caso 3: Geração de *Playbooks*

Este estudo de caso encerra o *pipeline*. O *ticket* chega pseudonimizado (Seção 1.5) e classificado (Seção 1.6); resta convertê-lo em uma recomendação estruturada de resposta.

A motivação operacional é direta. Paduraru et al. [PDS25] observam que as fases subsequentes à detecção, isto é, planejamento da mitigação, criação do *playbook* e execução, permanecem majoritariamente manuais, demoradas e propensas a inconsistências. Nguyen et al. [NMCMdO24] quantificam parte do problema: em 2023, as equipes levaram, em média, 63 dias para descobrir um incidente e mais 20 dias para implementar ações corretivas; Guduru [Gud25] cita uma média de 258 dias para identificar e conter uma violação de dados.

No ecossistema GT-LFI [GT-25b], o AutoClass-LFI automatiza a geração inicial desses procedimentos a partir de dois perfis complementares: **Linguagem Natural Estruturada** (Markdown), orientada ao analista e à governança, que detalha contexto, justificativas e passos e as etapas de resposta (identificação, contenção, erradicação, recuperação e lições aprendidas); e **Código de Automação** (Ansible YAML), que traduz as ações em tarefas idempotentes e parametrizadas voltadas à infraestrutura de servidores e *hosts*.

Essas cinco etapas correspondem, no vocabulário do NIST SP 800-61 Rev. 3 [NRSS25], às funções Detectar, Responder e Recuperar do CSF 2.0 [NIS24], e são mantidas aqui por serem a estrutura que os *playbooks* operacionais efetivamente adotam.

A conversão direta de alertas em código executável por LLMs introduz riscos operacionais severos. Por isso, este estudo de caso não avalia apenas a capacidade de sugestão do modelo. Apresenta-se uma inspeção qualitativa e detalhada de seis *playbooks* gerados, escolhidos para cobrir dois vetores taxonômicos distintos, com o objetivo de exibir os modos de falha característicos da tarefa. A amostra é deliberadamente pequena e não sustenta generalização estatística: seu papel é didático, no sentido de tornar visíveis erros que uma leitura superficial do artefato gerado não revelaria.

1.7.1. Anatomia de um *playbook* e geração assistida

Um *playbook* de resposta precisa conter, no mínimo, gatilho, pré-condições, passos ordenados, alvos, critérios de sucesso e pontos de decisão humana. Essa estrutura é consistente com Schlette et al. [SEC+24]: *playbooks* reais combinam procedimentos técnicos com fatores organizacionais e raramente são reutilizáveis sem adaptação ao contexto local.

O **Markdown** é o formato de consumo do analista: admite ressalvas e justificativas, mas não é executável nem verificável automaticamente. O **Ansible** converte cada passo em uma tarefa idempotente, propriedade especialmente importante quando o estado do ambiente é incerto. Em contrapartida, um *playbook* executável gerado por um modelo é, potencialmente, um vetor de ação inseguro.

A Listagem 1.3 apresenta um trecho de *playbook* Ansible para o incidente de DNS recursivo. Dois aspectos merecem atenção: a execução deve ser `manual`, refletindo a exigência de autorização humana antes de alterar configuração em produção; e o endereço aparece pseudonimizado, o que significa que a execução real requer a reidentificação controlada da Seção 1.5.1. O *playbook* gerado a partir de um dado pseudonimizado é um *modelo* de resposta, não uma ação diretamente aplicável.

```
name: Resposta a Incidente - DNS Recursivo Aberto (INC-2026-0001)
hosts: srv-dns-02.universidade.exemplo.br
become: yes
vars:
  trusted_networks: ["127.0.0.0/8", "192.168.1.0/24", "10.0.0.0/8"]
  dns_service_name: "named"
  backup_dir: "/var/backups/incident_inc_2026_0001"
tasks:
  # FASE 1: IDENTIFICACAO (Triage e Coleta)
  - name: "[Identificacao] Registrar conexoes ativas na porta 53"
    ansible.builtin.shell: "ss -tulpn | grep ':53'"
    register: active_dns_connections
    changed_when: false
  - name: "[Identificacao] Salvar estado inicial para analise forense"
    ansible.builtin.copy:
      content: "Conexoes Ativas: {{ active_dns_connections.stdout }}"
      dest: "{{ backup_dir }}/identificacao_estado_inicial.txt"
  # FASE 2: CONTENCAO (Mitigacao Imediata) - requer autorizacao manual
  - name: "[Contencao] Bloquear trafego UDP porta 53 de fontes externas"
    ansible.builtin.iptables:
      chain: INPUT
      protocol: udp
      destination_port: "53"
      jump: DROP
```

tags: firewall

Listagem 1.3. Trecho de playbook Ansible para servidor DNS recursivo aberto (exemplo sintético; versão completa no repositório do minicurso).

A geração assistida fundamenta-se em um desacoplamento em dois estágios: uma tarefa de inferência voltada à síntese analítica (Markdown) e outra dedicada à síntese de código operacional (Ansible YAML), isolando a formulação tática do procedimento de sua implementação sintática.

A engenharia de prompt adota *Role-Based System Prompting* combinado com *Structured Output Enforcement*: os *system prompts* delimitam o papel do agente, a taxonomia normativa e as fronteiras de restrição operacional. No canal de automação, o prompt exige código YAML válido sem preâmbulos, segregação de tarefas por fase de resposta, parametrização de variáveis, registro de *handlers* e preferência mandatória por módulos nativos e idempotentes do Ansible em detrimento de comandos genéricos de *shell*. Para mitigar comandos inseguros e ações cegas em produção, os prompts exigem que a coleta de evidências preceda qualquer comando de isolamento ou alteração de serviço, além de comentários elucidativos em cada bloco de tarefa.

O achado de Morais et al. [MQK⁺25] transfere-se diretamente a este contexto: o analista que conhece o objetivo de segurança, ainda que não domine a sintaxe do Ansible, é capaz de conduzir o refinamento do *prompt* até um resultado utilizável, desde que disponha de um mecanismo de validação a cada ciclo. A ferramenta não substitui o conhecimento de domínio, mas reduz a barreira de sintaxe.

1.7.2. Recuperação por similaridade

A geração de novos *playbooks* não é a única estratégia para apoiar a resposta a incidentes. O AI4SOAR [NMCMD024], por exemplo, adota uma abordagem baseada na reutilização: em vez de produzir um novo *playbook*, seleciona, dentre aqueles disponíveis em um repositório, o mais adequado ao alerta analisado, utilizando aprendizado de similaridade integrado à plataforma SOAR Shuffle. Guduru [Gud25], por sua vez, utiliza *playbooks* baseados no MITRE ATT&CK como contexto para a tomada de decisão e emprega modelos gráficos probabilísticos para validar as decisões resultantes.

Recuperar é preferível quando há repositório maduro e homologado, pois o procedimento já foi revisado, eliminando a categoria de riscos de alucinação; a limitação é a cobertura de alertas inéditos. **Gerar** é preferível quando o incidente é atípico ou o repositório é incipiente; a limitação é a confiabilidade, exigindo validação humana obrigatória. **A combinação**, que recupera o procedimento mais próximo e usa o modelo para adaptá-lo, é o desenho mais defensável, pois o modelo opera sobre um esqueleto já homologado, o que limita o espaço de erro.

1.7.3. Metodologia e resultados do estudo comparativo

O estudo de caso sobre o suporte a geração de *playbooks* foi conduzido com um corpus de 6 incidentes reais da base descrita na Seção 1.4.3, abrangendo CAT5 (Exploração de Vulnerabilidade), CAT3 (Negação de Serviço) processados pelo modelo *gemini-3.1-flash-lite*. A composição do corpus amostral é apresentada na Tabela 1.7.

Tabela 1.7. Corpus amostral para avaliação comparativa de *playbooks* gerados.

# / ID / Alerta	Categoria	Vetor / Protocolo	Foco no Markdown (NIST)	Escopo no Ansible
1-INC2022-127135	CAT5	TFTP Exposto (69/UDP)	Bloqueio de Borda + Desativação Local	package_facts,ufw,apt/yum
2-INC2022-138345	CAT5	NTP Amplification (123/UDP)	Desativação monlist + noquery	lineinfile,iptables,systemd
3-INC2022-140134	CAT5	SSDP/UPnP Exposto (1900/UDP)	Restrição de Interface + Hardening	service_facts,ufw,miniupnpd
4-INC2023-032613	CAT3	Varredura / Força Bruta SSH	Isolamento + Análise de Binários	ss,ps,dpkg -V,fail2ban
5-INC2023-062847	CAT3	Abuso DDoS Layer 7 (HTTPS)	Isolamento de Origem + Análise Forense	lsof,journalctl,find,fuser
6-INC2022-023007	CAT3	Botnet DDoS / Outbound Burst	Quarentena + Varredura de Malware	iptables,debsums,kill -9

Usaram-se dois perfis de prompt: (i) Geração Documental (Markdown), orientada às cinco etapas de resposta descritas na Seção 1.7; e (ii) Geração Executável (Ansible YAML), exigindo código idempotente estruturado em *tasks*, *handlers*, variáveis e salvaguardas operacionais. Os artefatos foram submetidos a uma matriz de comparação manual em seis dimensões: conformidade sintática, taxa de módulos nativos (em oposição a comandos crus de *shell*), alucinação de entidades, quebra de precedência forense, risco de ações destrutivas não autorizadas e falhas de escopo ou pseudocomandos.

Tabela 1.8. Auditoria quantitativa e qualitativa dos *playbooks* Ansible gerados.

# / Incidente	Cat.	Tasks Totais	% Módulos Nativos	Erros Sintaxe / Template	Alucinações de Contexto	Violação Forense	Risco Operacional Crítico	Falhas de Escopo / Pseudocomandos
1-INC2022-127135	CAT5	14	78,6%	1 (Jinja2 ternário)	Nenhuma	Não	purge: yes/xm -rf	0
2-INC2022-138345	CAT5	10	80,0%	1 (strftime inválido)	Nenhuma	Não	Regra inócua de firewall	0
3-INC2022-140134	CAT5	12	75,0%	0	Nenhuma	Não	Desinstalação forçada	1 (Falso positivo echo)
4-INC2023-032613	CAT3	14	50,0%	0	Hash cru como IP	Não	touch em chave SSH	1 (Token inválido no iptables)
5-INC2023-062847	CAT3	14	57,1%	1 (iptables recent)	Janela temporal	Não	fuser -k -9 443/tcp	1 (Inversão INPUT vs OUTPUT)
6-INC2022-023007	CAT3	14	50,0%	0	IP privado arbitrário	Não	Exclusão de authorized_keys	1 (kill -9 em arquivo)

Um resultado atravessa toda a Tabela 1.8 e merece destaque: a coluna **Violação Forense é “Não” nos seis incidentes**, sem exceção. Apesar dos erros de sintaxe, alucinações e riscos operacionais discutidos a seguir, a ordem de precedência entre coleta de evidência e contenção foi preservada em 100% dos casos, evidência de que a restrição imposta na engenharia de prompt (Seção 1.7.1) foi eficaz especificamente para essa dimensão, mesmo onde falhou em outras.

Consistência intra-grupo (CAT5, tríade de amplificação UDP). Em TFTP, NTP e SSDP, o modelo exibiu alta conformidade conceitual no Markdown, prescrevendo testes externos com `nmap` e contenção em firewall perimetral. No Ansible, a taxa de módulos nativos manteve-se elevada (75%–80%), mas a estratégia de erradicação variou: no NTP, o modelo aplicou edições cirúrgicas e idempotentes via `lineinfile` (`disable monitor` e `noquery`); já no TFTP, adotou uma abordagem destrutiva, expurgando pacotes (`purge: yes`) e removendo o diretório `/var/lib/tftpbboot` sem backup prévio; no SSDP, a falha equivalente foi uma desinstalação forçada do serviço `miniupnpd`, sem o mesmo cuidado de reversibilidade do caso NTP.

Degradação nos incidentes de botnet. Em INC2023-032613 (força bruta SSH, CAT3) e INC2022-023007 (botnet DDoS, CAT3), a taxa de módulos nativos caiu para 50%, com uso massivo de comandos de *shell* (`ps auxf`, `last`, `find`, `dpkg -V`). Essa abordagem garantiu boa coleta forense preliminar, mas introduziu fragilidades de execução, como o envio incorreto de nomes de arquivos ao utilitário `kill -9` em INC2022-023007. A leitura conjunta com a tríade de amplificação sugere uma regularidade plausível, embora não demonstrável com seis casos: quanto mais o incidente exige investigação em vez de reconfiguração, mais o modelo recorre a comandos genéricos, e menor é a garantia de idempotência.

Inversões semânticas e riscos críticos. O achado mais crítico do experimento é a dificuldade do LLM em interpretar o papel do host em reportes de abuso:

- **Inversão de vetor** (INC2023-062847, DDoS de camada 7): o alerta indicava a rede interna como origem do ataque; o *playbook* gerou regras na *chain INPUT* e executou `fuser -k -9 443/tcp`, derrubando o servidor web legítimo ao tentar conter um ataque de saída.
- **Erro de interpretação de token** (INC2023-032613): o modelo extraiu o *hash* produzido pela pseudonimização e o aplicou como endereço IP de origem no *iptables*, gerando erro fatal de compilação da regra. É o registro de “*hash* cru como IP” da Tabela 1.8 e ilustra um risco específico do *pipeline* deste capítulo: o pseudônimo, que protege o dado no EC1, torna-se um artefato sintaticamente plausível e semanticamente vazio para o gerador do EC3.
- **Auto-isolamento administrativo** (INC2022-023007): o modelo apagou `/root/.ssh/authorized_keys` e executou `iptables: flush: yes`, revogando o acesso administrativo remoto. No mesmo incidente, o IP privado arbitrário inserido pelo modelo (Tabela 1.8) não correspondia a nenhuma entidade do relato original.

1.7.4. Limites, riscos e validação humana

Esta é a subseção central do estudo de caso.

As taxas de omissão e de imprecisão medidas por Kramer et al. [KRN⁺25] na sumarização de incidentes, discutidas na Seção 1.3.3, transferem-se com folga para cá: se a operação autônoma falha naquela proporção em uma tarefa descritiva, é razoável esperar comportamento ao menos tão problemático na geração de *playbooks*, que é prescritiva e cujos erros têm consequência operacional direta. O mesmo estudo mostra, porém, que em uso colaborativo os modelos reduzem o esforço do analista e melhoram legibilidade e consistência. A variável decisiva é o *modo de uso*: uma saída apresentada como proposta a revisar produz ganho; a mesma saída, sem revisão, produz risco.

Catálogo de riscos. Os riscos de empregar LLMs no *pipeline* de um SOC podem ser organizados em quatro grupos [PJSS⁺25, OWA25]: **alucinações** exigem validação estrutural e revisão humana obrigatória antes de qualquer mitigação; **exposição de dados a APIs externas** é mitigada por sanitização e pseudonimização (papel do estudo de caso 1); **dependência de serviços externos** exige contingência via SOAR para análise humana quando a API está indisponível; e a **injeção de prompt**, risco subestimado, já que um relato de incidente é conteúdo controlado por terceiros, é mitigada por sanitização de entrada, gabaritos estruturados e validação com humano no laço.

Riscos específicos de *playbooks*. Três riscos são próprios desta tarefa. A **ação insegura**: um passo que interrompa serviço legítimo, bloqueie faixa excessivamente ampla ou destrua evidência antes da coleta. Por isso a coleta deve preceder a contenção e passos destrutivos devem ser sempre manuais. A **ausência de contexto operacional**: o modelo não conhece janelas de manutenção, dependências entre sistemas nem SLAs. A recuperação de procedimentos homologados (Seção 1.7.2) atenua parte do problema, porque o

procedimento recuperado já carrega decisões tomadas com esse contexto; preencher a lacuna de forma direta exigiria acoplar ao modelo, por RAG, o inventário de ativos e a base de configuração da organização, caminho que a Seção 1.8.5 discute como tendência. A **propagação de erro do estágio anterior**: uma classificação equivocada no EC2 produz um *playbook* corretamente gerado para a categoria errada, e o erro é silencioso, pois a saída é sintaticamente válida e coerente. É essa a razão pela qual o Módulo PoP posiciona a validação humana *entre* classificação e geração, não apenas ao final [GT-25b].

Limites verificáveis para agentes. À medida que a autonomia cresce, a validação posterior deixa de ser suficiente. O SENTINEL [SPA26] propõe verificação formal do comportamento do agente antes da execução, e não auditoria depois dela. A formulação do problema, mais que os números reportados, é a contribuição relevante. O IRCopilot [LZD⁺25] converge ao registrar perda de contexto, alucinações e preocupações de privacidade como obstáculos práticos em investigações de longo prazo.

1.7.5. Prática 3: geração assistida de *playbooks*

A prática opera sobre os incidentes classificados na Prática 2 e é deliberadamente *crítica*: o objetivo não é obter um bom *playbook*, mas aprender a identificar quando ele é ruim. Demonstra-se, na interface Web do AutoClass-LFI [GT-25b, GT-26a], a geração a partir de um incidente classificado, com saídas em Markdown e Ansible; a validação humana ocorre no Módulo PoP, onde o analista pode corrigir manualmente o *playbook* antes da execução.

A execução utiliza o *notebook* 03-playbooks.ipynb [SBS26]. O participante valida a saída sintática e semanticamente nos dois formatos, observando anomalias como erros de esquema, baixa taxa de módulos nativos, variáveis não inicializadas e quebras de precedência de caminhos, com o suporte de um auditor estático que sinaliza: (i) ações destrutivas sem salvaguardas; (ii) violação da ordem forense (contenção antes da coleta); e (iii) alucinação de entidades, topologias ou credenciais não fornecidas no *ticket*.

O analista responde ainda a quatro perguntas estruturadas por *playbook*: **Risco Operacional**: há comando cuja execução automática possa gerar indisponibilidade ou auto-bloqueio, sinalizado com ponto de decisão humana? **Alucinação de Entidades**: há arquivos, daemons, *templates* ou faixas de rede inseridos pelo modelo sem respaldo no relato? **Precedência Forense**: a extração de estado volátil, conexões e logs precede estritamente o isolamento de rede? **Contexto Organizacional**: que requisitos de governança (janelas de manutenção, SLAs, criticidade de ativos, autorizações) foram omitidos?

O resultado fecha o ciclo metodológico do minicurso: cada prática ilustra, isoladamente, uma etapa do mesmo *pipeline* que estrutura os estudos de caso, que são a pseudonimização (Prática 1), a classificação (Prática 2) e a geração de procedimento estruturado de resposta (Prática 3), com o humano no laço atuando como barreira mandatária em cada transição. Nas análises comparativas de cada estudo de caso, esse encadeamento é seguido de fato sobre o mesmo conjunto de incidentes; nas demonstrações ao vivo, cada prática opera sobre *tickets* independentes, escolhidos para exemplificar didaticamente o mecanismo daquela etapa específica.

1.7.6. Discussão

O *playbook* gerado por LLM deve ser entendido como apoio à decisão e à documentação, não como substituto do processo de resposta. Três consequências práticas decorrem dessa posição: o valor imediato mais defensável está na **redução do custo de documentação**, tarefa que equipes sob pressão sistematicamente adiam e em que os modelos demonstram ganho consistente no uso colaborativo [KRN⁺25]; o **ganho de padronização** é real e independente da qualidade absoluta do conteúdo, já que um repositório homogêneo é mais auditável e utilizável na formação do que documentos heterogêneos ainda que individualmente bem escritos; e o **ciclo de melhoria** é o que transforma a ferramenta em ativo institucional, pois no GT-LFI as correções feitas no Módulo PoP são incorporadas ao conhecimento do sistema [GT-25b], de modo que cada validação humana melhora a sugestão subsequente, preenchendo a lacuna de colaboração humano-IA identificada por Karunasingha et al. [KCN⁺25].

1.8. Desafios, Tendências e Conclusões

Esta seção consolida os limites observados ao longo do capítulo e aponta implicações práticas para a pesquisa, a operação e a formação.

1.8.1. Desafios de dados

Os quatro desafios de dados discutidos ao longo do capítulo formam um conjunto articulado, e não uma lista de itens independentes. A **escassez de dados reais rotulados** decorre de restrições de privacidade, de confidencialidade operacional e contratual, e é agravada pelo custo do rótulo, que exige julgamento de analista experiente. A **sensibilidade das informações compartilhadas por CSIRTs** torna a publicação de bases problemática de forma estrutural, e não apenas burocrática: para dados de vulnerabilidade, remover todos os identificadores destrói o conteúdo analítico do próprio conjunto [KLM⁺26]. O **desbalanceamento de classes** é a norma em bases operacionais e, como visto na Seção 1.4.3, torna a acurácia global uma métrica enganosa. A **falta de padronização de taxonomias** impede tanto a agregação de métricas entre organizações quanto a comparação de resultados entre trabalhos.

O efeito combinado é um viés sistemático na literatura: os resultados mais fortes são reportados em bases de dados próprias e não publicáveis, o que dificulta a replicação e favorece a repetição de números otimistas. Duas estratégias abordam parcialmente o problema. A primeira é a pseudonimização com compartilhamento sob acordo, adotada neste capítulo. A segunda é a geração sintética [GDB24], complementar à primeira, uma vez que dados pseudonimizados podem alimentar modelos que produzem conjuntos sintéticos publicáveis [KLM⁺26]. A equipe do GT-LFI registra a construção de uma base sintética de incidentes como prioridade para os próximos ciclos, justamente por seu impacto no compartilhamento e na replicabilidade [GT-26b].

1.8.2. Desafios de modelos e de avaliação

As taxas de alucinação medidas por Kramer et al. [KRN⁺25] e já discutidas na Seção 1.3.3 deveriam ser o ponto de partida de qualquer discussão sobre autonomia em resposta a incidentes. Elas não invalidam o uso, mas delimitam o modo de uso defensável. A esse

limite soma-se a inconsistência entre execuções: a mesma entrada pode produzir saídas diferentes, o que é problemático em um domínio no qual a rastreabilidade da decisão importa. As mitigações conhecidas são temperatura baixa, critérios de parada por estabilização e voto majoritário entre execuções, todas com custo em latência ou em chamadas de API.

Um terceiro limite é a dependência de contexto. Modelos não conhecem a topologia, as janelas de manutenção nem a criticidade dos ativos da organização. O RAG mitiga o problema, mas o transfere para a qualidade e a atualização da base recuperada. Some-se a isso a explicabilidade: o diagnóstico de Nadeem et al. [NVC⁺23] permanece válido, no sentido de que a explicabilidade praticada serve ao pesquisador e não ao operador. Para o analista de SOC, uma atribuição de importância de *features* raramente responde à pergunta que ele precisa fazer, que é se deve ou não agir.

Por fim, avaliar um sistema de IA em ambiente operacional esbarra em três obstáculos que se reforçam. A referência é o julgamento humano, que diverge de si mesmo em cerca de 18,5% dos casos [JFSAM25]. O custo do erro é assimétrico entre categorias, algo que métricas simétricas não capturam. E o ambiente muda ao longo do tempo, de modo que o desempenho medido hoje não se sustenta necessariamente amanhã, fenômeno tratado em [PJSS⁺25] sob o conceito de monitoramento de desvio (*drift*).

1.8.3. Privacidade, conformidade e soberania de dados

Primeiro, **pseudonimização não é anonimização**: como o dado pseudonimizado permanece reversível para quem detém a chave e o mapeamento, ele continua sendo dado pessoal perante a LGPD [Bra18, ANP23], e equipes que tratam os dois conceitos como equivalentes assumem risco de conformidade não percebido. Segundo, a **dependência de provedores externos** introduz risco de privacidade, dependência de jurisdição estrangeira e exposição a decisões unilaterais de fornecedores sobre disponibilidade, preço ou descontinuação de modelos; tratamos o tema sob a ótica da soberania de dados e da cadeia de suprimentos de modelos em [PJSS⁺25], e Almeida et al. [APS⁺25] o abordam do ponto de vista da conformidade e do custo. Terceiro, a **execução local é a resposta técnica** disponível, com custo honestamente delimitado: acurácia em torno de 60% para modelos abertos de 8B a 20B, contra mais de 90% para modelos comerciais [APS⁺25], com requisitos de hardware mensuráveis [PSA⁺25]. O desenho em dois níveis proposto na Seção 1.6.1, com triagem local e escalonamento seletivo, é a acomodação prática entre esses dois regimes.

1.8.4. Governança de IA e o fator humano

Esta subseção responde à pergunta prática que atravessa o capítulo: *quanta autonomia delegar e a quê*.

Princípios de governança. O NIST AI Risk Management Framework [NIS23] organiza a gestão de risco de sistemas de IA em quatro funções, aplicáveis diretamente ao contexto de um SOC [PJSS⁺25]. **Governar** permeia todas as demais e estabelece políticas, estruturas de responsabilidade, processos de documentação e mecanismos de supervisão ao longo do ciclo de vida do sistema. **Mapear** estabelece o contexto para enquadrar riscos, identificar os objetivos dos sistemas de detecção e caracterizar os impactos sobre a ope-

ração, os analistas e a infraestrutura. **Medir** emprega métodos quantitativos e qualitativos para analisar e monitorar riscos, com testes, verificações e validações dos modelos antes e durante a operação. **Gerir** aloca recursos aos riscos mapeados, prioriza por impacto e probabilidade e estabelece monitoramento pós-implantação. Normas complementares da ITU-T podem apoiar a governança em telecomunicações, entre elas a Y.3172, a M.3080 e a X.1060 [PJSS+25].

Níveis de automação por função de SOC. A Tabela 1.9 apresenta o mapeamento das funções do modelo de SOC descrito na Seção 1.2.2 em três níveis de automação [PJSS+25]. Ela converte a discussão abstrata sobre autonomia em uma decisão por função. A leitura revela um padrão claro: a automação total concentra-se nas funções de *transporte e preparação* do dado, ao passo que todas as funções que envolvem *juízo* são parciais ou manuais. As três tarefas tratadas nos estudos de caso deste capítulo situam-se integralmente na faixa parcial.

Tabela 1.9. Mapeamento das funções de um SOC em níveis de automação, adaptado de [PJSS+25]. Total: opera sem intervenção humana durante a execução. Parcial: requer supervisão, validação ou aprovação em pontos críticos. Manual: permanece sob controle humano.

Função	Nível	Intervenção humana
Sensores de telemetria	Total	Configuração inicial
Agentes de coleta	Total	Configuração inicial
Ingestão em fila de mensagens	Total	Configuração inicial
Normalização de dados	Total	Configuração dos decodificadores
Armazenamento e indexação	Total	Definição de políticas de retenção
Análise em tempo real	Parcial	Criação e ajuste de regras
Deteção avançada	Parcial	Criação e ajuste de assinaturas
Aprendizado de máquina	Parcial	Preparação de dados e retreinamento
Visualização e <i>dashboards</i>	Parcial	Customização de painéis
Integração com IA/LLM	Parcial	Configuração e supervisão contínua
SOAR	Parcial	Aprovação das ações de mitigação
<i>Threat intelligence</i>	Parcial	Validação de indicadores
Equipes e operação	Manual	Todas as decisões

O papel do fator humano. O conceito de humano no laço (*human-in-the-loop*) é o eixo tanto da metodologia operacional em [PJSS+25] quanto do desenho do GT-LFI [GT-25b], e é enfatizado pelo NIST SP 1270 como prática central de mitigação de riscos de IA [SVG+22]. Vale explicitar por que ele não é apenas uma salvaguarda defensiva. Primeiro, o analista traz conhecimento contextual que não pode ser capturado por um modelo: a infraestrutura específica, os padrões normais de tráfego, o contexto de negócio e os requisitos regulatórios. Em segundo lugar, a redução de *alert fatigue* por meio de IA não visa eliminar a participação humana, mas sim realocá-la para a análise aprofundada de incidentes críticos. Terceiro, e mais importante para o argumento deste capítulo, a validação humana é a única fonte de sinal de correção disponível: no ciclo do GT-LFI, as correções feitas no Módulo PoP são incorporadas ao conhecimento do sistema, o que responde diretamente à lacuna de colaboração humano-IA apontada

por Karunasingha et al. [KCN⁺25]. Uma condição frequentemente esquecida é que o analista precisa *compreender* o sistema que supervisiona: sem documentação adequada e capacitação, a supervisão degenera em aprovação automática, e o controle torna-se nominal [PJSS⁺25].

1.8.5. Tendências

A atividade recente concentra-se em seis linhas. A primeira é o **RAG sobre bases de CTI e de contexto organizacional**, com o deslocamento do RAG documental simples para arquiteturas multiestágio que combinam contexto topológico em grafo com a recuperação de procedimentos [PJSS⁺25, ÖVT26, HAL26]. A segunda é a **geração sintética de incidentes**, que endereça a escassez de dados e viabiliza o compartilhamento [GDB24, KMC⁺26], com o cuidado de que os dados sintéticos herdem os vieses do gerador. A terceira é os **modelos especializados e o ajuste fino para segurança**: o pré-treinamento contínuo adaptado ao domínio [KSC⁺25] reduz a lacuna entre modelos locais e comerciais sem abrir mão da soberania de dados, e Almeida et al. [APS⁺25] apontam o ajuste fino eficiente em parâmetros, como LoRA, entre suas direções futuras. A quarta é a **integração entre LLMs e plataformas SOAR**, por meio de recuperação por similaridade [NMCMdO24] ou de orquestração agêntica [ÖVT26, Gud25], com o Model Context Protocol emergindo como uma camada de padronização da comunicação entre o modelo e as ferramentas [PJSS⁺25]. A quinta é a dos **agentes autônomos supervisionados com limites verificáveis** [SPA26], linha que distingue propostas defensáveis de propostas apenas ambiciosas. A sexta é a **colaboração humano-IA como objeto de projeto**: a evidência de que o modo de uso importa mais do que a capacidade do modelo [KRN⁺25] desloca o problema da engenharia de modelos para a engenharia de interação, campo em que a lacuna identificada por Karunasingha et al. [KCN⁺25] permanece aberta.

1.8.6. Aprender com incidentes: formação e capacitação

Há uma dimensão da resposta a incidentes que a discussão técnica costuma deixar de fora, e que o acervo deste minicurso permite tratar: o incidente real, uma vez anonimizado, é um insumo pedagógico de altíssimo valor.

O diagnóstico que motiva essa linha é o do GT-LFI [GT-25b]: cursos tradicionais de formação em segurança priorizam conteúdos conceituais e exercícios descontextualizados, o que distancia a formação das rotinas e das pressões reais das equipes de resposta. O resultado é que novos profissionais ingressam no mercado sem experiência suficiente para reconhecer padrões de comportamento malicioso, classificar incidentes com precisão ou conduzir respostas sob pressão, o que agrava o déficit global descrito na abertura do capítulo.

A resposta do GT-LFI consiste em converter o ciclo operacional em um ciclo pedagógico. Incidentes tratados por analistas no Módulo PoP podem ser selecionados para alimentar trilhas e desafios na plataforma gamificada, criando um fluxo contínuo entre prática, aprendizado e automação. A avaliação com residentes forneceu evidência quantitativa desse ciclo: na primeira rodada, com 35 respondentes, registraram-se 508 lições concluídas, 235 quizzes submetidos, 173 módulos e 79 trilhas concluídas; na segunda

rodada, com desenho de pré-teste e pós-teste sobre dez questões, a taxa média de acerto subiu de 67,9% para 91,7% [GT-26b]. Os autores registram as limitações do resultado, isto é, redução amostral entre as etapas, ausência de grupo de controle e amostra pequena, o que lhe confere caráter descritivo e não inferencial.

Um achado pedagógico específico dialoga diretamente com o estudo de caso 2: entre as quatro trilhas avaliadas, a de classificação de incidentes concentrou o maior número de tentativas (94), o menor escore médio (65,27) e a menor taxa de aprovação (54,26%), ao passo que a trilha de *playbooks* para DDoS alcançou escore de 83,80 e aprovação de 82,86% [GT-26b]. Classificar corretamente é difícil também para humanos em formação, o que contextualiza os limites observados nos modelos automáticos e reforça a ideia de que a tarefa é intrinsecamente ambígua, e não apenas tecnicamente desafiadora. Complementarmente, o HIKARI [BVL+25, PJSS+25] oferece o ambiente de simulação que essas trilhas pressupõem, reproduzindo, de forma controlada e anônima, as condições de trabalho de um analista em um ambiente do tipo SIEM.

1.8.7. Conclusões

Este capítulo percorreu, do fundamento normativo à prática executável, o uso de Inteligência Artificial na resposta a incidentes. Encerramos com cinco conclusões.

A IA não substitui o processo de resposta a incidentes. Ela amplia a capacidade de triagem, análise, documentação e recomendação quando integrada a *frameworks* consolidados, dados confiáveis, anonimização adequada e validação humana. A evidência sustenta essa formulação de maneira precisa: modelos operando de forma autônoma falham em proporções incompatíveis com uso operacional; os mesmos modelos em uso colaborativo produzem ganho consistente [KRN+25].

A metodologia importa tanto quanto o modelo. O conjunto de experimentos discutidos mostra que a combinação de taxonomia estruturada com heurística iterativa de *prompting* melhora a consistência da classificação de forma sistemática, em todas as arquiteturas avaliadas [SLK+25, SLA+25]. No mesmo sentido, um único exemplo rotulado por categoria pode dobrar o F1 de um classificador [JFSAM25], resultado que este capítulo reproduz sobre a própria base, com a acurácia global passando de 68,4% para 89,7% (Seção 1.6.4). Ambas são intervenções de baixo custo e alto retorno, disponíveis para qualquer equipe.

Privacidade é requisito de projeto, não etapa posterior. A pseudonimização não é um passo burocrático que antecede o trabalho real: é a condição que torna possível operar sobre dados reais. E a decisão de projeto que a viabiliza sem destruir a utilidade, isto é, pseudonimização determinística com preservação do tipo da entidade, é o que permite que todo o restante do *pipeline* funcione [KLM+26]. Quando a política de dados exige que os dados não saiam da organização, a execução local de modelos é tecnicamente viável, com custo de acurácia conhecido e quantificado [APS+25, PSA+25].

A validação humana também é uma fonte de sinal. O desenho que distingue as propostas maduras é aquele em que a correção do analista realimenta o sistema. É essa realimentação que responde à lacuna de colaboração humano-IA [KCN+25] e converte a ferra-

menta de um produto estático em um ativo institucional que melhora com o uso [GT-25b].

O incidente real é um ativo de dupla função. Uma vez tratado com o devido cuidado, o mesmo material que alimenta a operação também alimenta a formação. Fechar esse ciclo, da operação à sala de aula e de volta, é uma contribuição direta que a comunidade brasileira de segurança pode extrair do trabalho apresentado neste capítulo. O material prático que acompanha este capítulo, publicado em <https://github.com/orgs/sbseg26-mc1> [SBS26], é um passo nessa direção.

Agradecimentos

Os trabalhos que sustentam este capítulo contaram com apoio parcial da CAPES, código de financiamento 001; da RNP, por meio do Programa Hackers do Bem e do GT-LFI (*Learn From Incidents*); da FAPERGS, por meio dos termos de outorga 24/2551-0001368-7 e 24/2551-0000726-1; do CNPq (processos 409743/2025-9 e 408432/2024-1); e da ANATEL, por meio do Termo de Execução Descentralizada com o ITA. Agradecemos aos CSIRTs e aos Pontos de Presença parceiros pela cessão dos dados que tornaram possíveis os estudos de caso.

Referências

- [AAM22] Bushra A. Alahmadi, Louise Axon, and Ivan Martinovic. 99% false positives: A qualitative study of SOC analysts' perspectives on security alarms. In *31st USENIX Security Symposium*, pages 2783–2800, 2022.
- [ABDL19] Adham Albakri, Eerke Boiten, and Rogério De Lemos. Sharing cyber threat intelligence under the general data protection regulation. In *Privacy Technologies and Policy (APF)*. Springer, 2019.
- [AI 25a] AI Horizon Labs. SecLINC: Security incident labeling with LLMs. <https://github.com/AI-Horizon-Labs/SecLINC>, 2025.
- [AI 25b] AI Labs 4 All. FrameworkPE: Prompt engineering framework for incident classification. <https://github.com/AI4All/FrameworkPE>, 2025.
- [Ano26] AnonShield. AnonShield: Repositório do código-fonte. <https://github.com/AnonShield/anonshield>, 2026.
- [ANP23] ANPD. Estudo Preliminar: Anonimização e Pseudonimização para a Proteção de Dados Pessoais. Autoridade Nacional de Proteção de Dados, Brasília, 2023.
- [APS⁺25] Gefté Almeida, Marcio Pohlmann, Alex Severo, Diego Kreutz, Tiago Heinrich, and Lourenço Alves Pereira Júnior. On-premise SLMs vs. commercial LLMs: Prompt engineering and incident classification in SOCs and CSIRTs. *arXiv preprint arXiv:2511.14908*, 2025.
- [BEG⁺25] Carolina Tompsen Bandel, João Pedro Ramires Esteves, Kalian Pereira Guerra, Leandro Márcio Bertholdo, Diego Kreutz, and Rodrigo Sanches Miani. Anonimização de incidentes de segurança com reidentificação controlada. In *Anais do XXV Simpósio Brasileiro de Cibersegurança (SBSeg)*, pages 97–113. SBC, 2025.
- [BMR⁺20] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1877–1901, 2020.
- [Bra18] Brasil. Lei n. 13.709, de 14 de agosto de 2018. lei geral de proteção de dados pessoais (LGPD), 2018.

- [BVL⁺25] Sidnei Barbieri, Gustavo Viana, Pedro Lima, Caio Carvalho, Eduardo Oliveira, and Lourenço Alves Pereira Júnior. HIKARI – real challenges, hidden threats: A gamified threat hunting platform. In *Anais do XXV Simpósio Brasileiro de Cibersegurança (SBSeg)*. SBC, 2025.
- [CER25] CERT.br. Incidentes notificados ao CERT.br. <https://stats.cert.br/incidentes/>, 2025.
- [CTJ24] Junyi Chen, Jiahe Tian, and Yi Jin. Self-hint prompting improves zero-shot reasoning in large language models via reflective cycle. In *46th Annual Conference of the Cognitive Science Society*, 2024.
- [CVE26] CVE Details. Browse CVE vulnerabilities by date. <https://www.cvedetails.com/>, 2026.
- [DHA⁺23] Savino Dambra, Yufei Han, Simone Aonzo, Platon Kotzias, Antonino Vitale, Juan Caballero, Davide Balzarotti, and Leyla Bilge. Decoding the secrets of machine learning in windows malware classification: A deep dive into datasets, features, and model performance. In *ACM CCS*, pages 60–74, 2023.
- [Dwo06] Cynthia Dwork. Differential privacy. In *33rd International Colloquium on Automata, Languages and Programming (ICALP)*, pages 1–12, 2006.
- [ENI18] ENISA. Reference Incident Classification Taxonomy. European Union Agency for Cybersecurity, 2018.
- [FIR19] FIRST. CSIRT Services Framework, version 2.1. https://www.first.org/standards/frameworks/csirts/csirt_services_framework_v2.1, 2019.
- [GDB24] Haula Sani Galadima, Cormac Doherty, and Rob Brennan. Towards LLM-based synthetic dataset generation of cyber incident response process logs. In *International Conference on Cyber Resilience and Cyber Intelligence (Cyber-RCI)*, pages 1–4, 2024.
- [GT-25a] GT-LFI. ANON-LFI: Anonimização de incidentes de segurança. <https://github.com/gt-rnp-lfi/anon-lfi>, 2025.
- [GT-25b] GT-LFI. White Paper do Projeto P&D Hackers do Bem: GT-LFI (Learn From Incidents). Technical report, Rede Nacional de Ensino e Pesquisa (RNP), 2025. Disponível em <https://gt-rnp-lfi.github.io/home/>.
- [GT-26a] GT-LFI. AutoClass-LFI: Classificação automática de incidentes e geração de *Playbooks*. <https://github.com/gt-rnp-lfi/lfi-autoclass>, 2026.
- [GT-26b] GT-LFI. Relatório Final da Extensão do Ciclo 2 do GT-LFI (Learn From Incidents). Technical report, Rede Nacional de Ensino e Pesquisa (RNP), 2026. Disponível em <https://gt-rnp-lfi.github.io/home/>.
- [Gud25] Sandhya Guduru. Autonomous cyber defense: LLM-powered incident response with LangChain and SOAR integration. *International Journal of Computer Science and Information Technology Research (IJCSITR)*, 6(1):72–82, 2025.
- [HAL26] Kim Hammar, Tansu Alpcan, and Emil C. Lupu. Incident response planning using a lightweight large language model with reduced hallucination. In *NDSS*, 2026.
- [HČ22] Martin Husák and Milan Čermák. SoK: Applications and challenges of using recommender systems in cybersecurity incident handling and response. In *17th International Conference on Availability, Reliability and Security (ARES)*, pages 1–10, 2022.
- [Ibr19] Marina Danchovsky Ibrishimova. Cyber incident classification: Issues and challenges. In *Advances on P2P, Parallel, Grid, Cloud and Internet Computing*, pages 469–477. Springer, 2019.
- [ISC23] ISC2. ISC2 reveals workforce growth but record-breaking gap of 4 million cybersecurity professionals. <https://www.isc2.org/insights/2023/10/>, 2023.

- [JFSAM25] Sebastião Alves de Jesus Filho, Alvaro Santana Santos, Rafael Dias Araújo, and Rodrigo Sanches Miani. Applying LLMs to the classification of cybersecurity incident tickets in a few-shot scenario. In *24th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 960–965. IEEE, 2025.
- [KCN⁺25] Navodika Karunasingha, Mohan Baruwat Chhetri, Surya Nepal, Cecile Paris, and Salil S. Kanhere. SoK: AI support for analyst situation awareness in security operation centres. In *European Symposium on Usable Security (EuroUSEC)*, pages 151–163, 2025.
- [KLM⁺26] Cristhian Kapelinski, Douglas Lautert, Beatriz Machado, Diego Kreutz, and Isadora Garcia Ferrão. AnonShield: Scalable on-premise pseudonymization for CSIRT network vulnerability data. In *Anais Estendidos do XLIV Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC), Salão de Ferramentas*. SBC, 2026.
- [KLMK25] Cristhian Kapelinski, Douglas Lautert, Beatriz Machado, and Diego Kreutz. AnonLFI 2.0: Extensible architecture for PII pseudonymization in CSIRTs with OCR and technical recognizers. In *Workshop de Redes Regionais e Cibersegurança (WRSeg/ERRC)*. SBC, 2025. arXiv:2511.15744.
- [KMC⁺26] Shravya Kanchi, Neal Mangaokar, Aravind Cheruvu, Sifat Muhammad Abdullah, Shirin Nilizadeh, Atul Prakash, and Bimal Viswanath. Taming data challenges in ml-based security tasks using generative ai. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS)*, pages 788–804, 2026.
- [KRN⁺25] Diana Kramer, Lambert Rosique, Ajay Narotam, Elie Bursztein, Patrick Gage Kelley, Kurt Thomas, and Allison Woodruff. Integrating large language models into security incident response. In *Twenty-First Symposium on Usable Privacy and Security (SOUPS)*, pages 133–148. USENIX Association, 2025.
- [KSC⁺25] Paul Kassianik, Baturay Saglam, Alexander Chen, Blaine Nelson, Anu Vellore, et al. Llama-3.1-FoundationAI-SecurityLLM-Base-8B technical report. Technical report, Foundation AI, Cisco, 2025.
- [LLV07] Ninghui Li, Tiancheng Li, and Suresh Venkatasubramanian. t-closeness: Privacy beyond k-anonymity and l-diversity. In *IEEE 23rd International Conference on Data Engineering (ICDE)*, pages 106–115, 2007.
- [LPP⁺20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [LTHJ24] Yitian Li, Jiahe Tian, Hao He, and Yi Jin. Hypothesis testing prompting improves deductive reasoning in large language models. *arXiv preprint arXiv:2405.06707*, 2024.
- [LZD⁺25] Xihuan Lin, Jie Zhang, Gelei Deng, Tianzhe Liu, Tianwei Zhang, Riqing Chen, and Qing Guo. IRCopilot: Automated incident response with large language models. *arXiv preprint arXiv:2505.20945*, 2025.
- [Mic18] Microsoft. Presidio: Data protection and de-identification SDK. <https://github.com/microsoft/presidio>, 2018.
- [MIT25] MITRE. ATT&CK Matrix for Enterprise. <https://attack.mitre.org/>, 2025.
- [MKGv07] Ashwin Machanavajjhala, Daniel Kifer, Johannes Gehrke, and Muthuramakrishnan Venkitasubramaniam. l-diversity: Privacy beyond k-anonymity. *ACM Transactions on Knowledge Discovery from Data*, 1(1):3:1–3:52, 2007.
- [MLKK25] Beatriz Machado, Douglas Lautert, Cristhian Kapelinski, and Diego Kreutz. Structured extraction of vulnerabilities in OpenVAS and Tenable WAS reports using LLMs. In *Workshop de Redes Regionais e Cibersegurança (WRSeg/ERRC)*. SBC, 2025.

- [MQK⁺25] Tiago W. Morais, Silvio Ereno Quincozes, Diego Kreutz, Juliano F. Kazienko, and Vagner E. Quincozes. Análise exploratória de criação de regras do WAF ModSecurity utilizando cinco LLMs distintos. In *Anais do XXV Simpósio Brasileiro de Cibersegurança (SBSeg), Artigos Curtos*. SBC, 2025.
- [NIS23] NIST. Artificial Intelligence Risk Management Framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, 2023.
- [NIS24] NIST. The NIST Cybersecurity Framework (CSF) 2.0. NIST Cybersecurity White Paper CSWP 29, National Institute of Standards and Technology, 2024.
- [NMCMD024] Manh-Dung Nguyen, Wissam Mallouli, Ana Rosa Cavalli, and Edgardo Montes de Oca. AI4SOAR: A security intelligence tool for automated incident response. In *19th International Conference on Availability, Reliability and Security (ARES)*, pages 170:1–170:8, 2024.
- [NRSS24] Alexander Nelson, Sanjay Rekhi, Murugiah Souppaya, and Karen Scarfone. Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile (Initial Public Draft). NIST Special Publication 800-61r3 ipd, National Institute of Standards and Technology, Gaithersburg, MD, 2024.
- [NRSS25] Alexander Nelson, Sanjay Rekhi, Murugiah Souppaya, and Karen Scarfone. Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile. NIST Special Publication 800-61r3, National Institute of Standards and Technology, Gaithersburg, MD, 2025.
- [NS08] Arvind Narayanan and Vitaly Shmatikov. Robust de-anonymization of large sparse datasets. In *IEEE Symposium on Security and Privacy*, pages 111–125, 2008.
- [NVC⁺23] Azqa Nadeem, Daniël Vos, Clinton Cao, Luca Pajola, Simon Dieck, Robert Baumgartner, and Sicco Verwer. SoK: Explainable machine learning for computer security applications. In *IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, pages 221–240, 2023.
- [OAS23] OASIS. CACAO Security Playbooks Version 2.0. OASIS Committee Specification, 2023.
- [Oll25] Ollama. Ollama: Get up and running with large language models locally. <https://ollama.com>, 2025.
- [ÖVT26] Enes Özgözler, Asaf Varol, and İhsan Tanrıverdi. Event-driven agentic SOC (ED-ASOC): Supervisor-based LLM framework for dynamic incident response and SOAR orchestration. In *ISDFS*, pages 1–5, 2026.
- [OWA25] OWASP Foundation. OWASP top 10 for LLM applications. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>, 2025.
- [PDS25] Ciprian Paduraru, Bogdan Dumitru, and Alin Stefanescu. Automated generation of cybersecurity response playbooks via large language models. *Procedia Computer Science*, 270:2987–2996, 2025.
- [PJS25a] Lourenço Alves Pereira Júnior and Murray Evangelista de Souza. Relatório Parcial n. 1: Investigação de Ameaças Cibernéticas no Setor de Telecomunicações. Ted ita/anatel, eixo 3, cibersegurança em telecomunicações, Instituto Tecnológico de Aeronáutica (ITA), Divisão de Ciência da Computação, São José dos Campos, SP, 2025.
- [PJS25b] Lourenço Alves Pereira Júnior and Murray Evangelista de Souza. Relatório Parcial n. 2: Investigação das Técnicas de Inteligência Artificial em Cibersegurança. Ted ita/anatel, eixo 3, cibersegurança em telecomunicações, Instituto Tecnológico de Aeronáutica (ITA), Divisão de Ciência da Computação, São José dos Campos, SP, 2025.

- [PJSS⁺25] Lourenço Alves Pereira Júnior, Murray Evangelista de Souza, Wagner Comin Sonaglio, Sidnei Barbieri, and Ágney Lopes Roth Ferraz. Relatório Parcial n. 3: Metodologia para Aplicação de Inteligência Artificial na Detecção e Resposta a Incidentes de Cibersegurança em Redes de Telecomunicações. Ted ita/anatel, eixo 3, cibersegurança em telecomunicações, Instituto Tecnológico de Aeronáutica (ITA), Divisão de Ciência da Computação, São José dos Campos, SP, 2025.
- [PKLK14] Fabian Prasser, Florian Kohlmayer, Ronald Lautenschläger, and Klaus A. Kuhn. ARX: A comprehensive tool for anonymizing biomedical data. In *AMIA Annual Symposium Proceedings*, pages 984–993, 2014.
- [PSA⁺25] Marcio Pohlmann, Alex Severo, Gefté Almeida, Diego Kreutz, Tiago Heinrich, and Lourenço Alves Pereira Júnior. Temperature in SLMs: Impact on incident categorization in on-premises environments. *arXiv preprint arXiv:2511.19464*, 2025.
- [SBS26] SBSeg 2026 Minicurso 1. Material complementar do minicurso IA aplicada à resposta a incidentes: *Notebooks*, dados sintéticos, *Prompts* e *Playbooks*. <https://github.com/orgs/sbseg26-mc1>, 2026.
- [SEC⁺24] Daniel Schlette, Philip Empl, Marco Caselli, Thomas Schreck, and Günther Pernul. Do you play it by the books? a study on incident response playbooks and influencing factors. In *IEEE Symposium on Security and Privacy (S&P)*, pages 3625–3643, 2024.
- [SIB⁺24] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, et al. The prompt report: A systematic survey of prompting techniques. *arXiv preprint arXiv:2406.06608*, 2024.
- [SLA⁺25] Alex Sandre Pinheiro Severo, Douglas Paim Lautert, Gefté Alcantara de Almeida, Diego Kreutz, Godinho Rodrigo, Lourenço Alves Pereira Júnior, and Leandro Márcio Bertholdo. LLMs e engenharia de prompt para classificação automatizada de incidentes em SOCs. In *Anais Estendidos do XXV Simpósio Brasileiro de Cibersegurança (SBSeg)*, *Trilha de Indústria e Inovação*. SBC, 2025.
- [SLK⁺25] Alex Sandre Pinheiro Severo, Douglas Paim Lautert, Diego Kreutz, Leandro Márcio Bertholdo, Marcio Pohlmann, and Silvio Ereno Quincozes. Categorização de incidentes de segurança utilizando engenharia de prompts em LLMs. In *Anais do XXV Simpósio Brasileiro de Cibersegurança (SBSeg)*, pages 256–272. SBC, 2025.
- [SPA26] Tanvir H. Sardar, Bishwajeet Pandey, and Laura Aldasheva. LLM-powered autonomous security agents for next-generation cyber defence. In *5th IEEE International Conference on AI in Cybersecurity (ICAIC)*. IEEE, 2026.
- [SVG⁺22] Reva Schwartz, Apostol Vassilev, Kristen Greene, Lori Perine, Andrew Burt, and Patrick Hall. Towards a Standard for Identifying and Managing Bias in Artificial Intelligence. NIST Special Publication 1270, National Institute of Standards and Technology, 2022.
- [Swe00] Latanya Sweeney. Simple demographics often identify people uniquely. Data Privacy Working Paper 3, Carnegie Mellon University, 2000.
- [Swe02] Latanya Sweeney. k-anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5):557–570, 2002.
- [TKM15] Matthias Templ, Alexander Kowarik, and Bernhard Meindl. Statistical disclosure control for micro-data using the R package sdcMicro. *Journal of Statistical Software*, 67(4):1–36, 2015.
- [VSC⁺21] André Varanda, Leonel Santos, Rogério Luís de C. Costa, Adriana Oliveira, and Carlos Rabadão. Log pseudonymization: Privacy maintenance in practice. *Journal of Information Security and Applications*, 63:103021, 2021.
- [WDS⁺19] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, et al. HuggingFace’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019.

- [XFAM01] Jun Xu, Jinliang Fan, Mostafa H. Ammar, and Sue B. Moon. On the design and performance of prefix-preserving IP traffic trace anonymization. In *ACM SIGCOMM Internet Measurement Workshop (IMW)*, pages 263–266, 2001.
- [YZY⁺23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [ZLX⁺23] Chuanyang Zheng, Zhengying Liu, Enze Xie, Zhenguo Li, and Yu Li. Progressive-hint prompting improves reasoning in large language models. *arXiv preprint arXiv:2304.09797*, 2023.
- [ZZL⁺23] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.